

Hybrid noise-resistant technique for malware classification

Received: 17 July 2025

Accepted: 5 June 2026

Published online: 09 July 2026

Cite this article as: Shah S.S.H., Shah S.S.H., Hamed A. *et al.* Hybrid noise-resistant technique for malware classification. *Sci Rep* (2026). <https://doi.org/10.1038/s41598-026-57376-7>

Syed Shahid Hameed Shah, Syed Shakir Hameed Shah, Ahmed Hamed, Atta ur Rehman Khan & Mathew Nicho

We are providing an unedited version of this manuscript to give early access to its findings. Before final publication, the manuscript will undergo further editing. Please note there may be errors present which affect the content, and all legal disclaimers apply.

If this paper is publishing under a Transparent Peer Review model then Peer Review reports will publish with the final article.

ARTICLE IN PRESS

Hybrid Noise-Resistant Technique for Malware Classification

Syed Shahid Hameed Shah^{1,*}, Syed Shakir Hameed Shah², Ahmed Hamed³, Atta ur Rehman Khan⁴, and Mathew Nicho^{5,6}

¹Department of Computer Science, Sohar University, Oman

²Network and System Engineer, West Herts College, Watford, London UK

³School of Engineering, Applied Science, and Technology, Canadian University Dubai

⁴College of Engineering and Information Technology, Ajman University, Ajman UA

⁵Research and Innovation Centre, Rabdan Academy, Abu Dhabi, United Arab Emirates

⁶UQ Cyber Research Centre, University of Queensland, Australia.

*shahidccie@gmail.com

+these authors contributed equally to this work

ABSTRACT

Visualization-based malware detection has recently gained significant attention for binary and multiclass malware classification using machine learning and deep learning techniques. However, existing visualization-based frameworks still face several important limitations, including insufficient robustness evaluation, limited cross-dataset validation, restricted malware diversity, and difficulty distinguishing visually similar and noise-sensitive malware families. In many cases, the visual similarity between malware classes and the presence of perturbations negatively affect feature extraction quality, leading to degraded classification performance and reduced generalization capability. To address these challenges, this study proposes a novel hybrid malware representation framework that integrates Convolutional Autoencoder (CAE)-based latent structural learning with Local Binary Pattern (LBP)-based texture feature extraction for robust malware classification. To the best of our knowledge, this study represents one of the first comprehensive investigations of hybrid latent-texture representation learning within a memory-forensics malware visualization setting while jointly addressing robustness, perturbation resilience, scalability, and cross-dataset generalization through a unified evaluation framework. The proposed framework combines global hierarchical representations learned through CAE with fine-grained local texture descriptors extracted using LBP to improve the discrimination of visually similar malware families and enhance robustness against noisy visualization conditions. The extracted features are subsequently evaluated using multiple machine learning classifiers, where XGBoost achieved the highest performance with an accuracy of 99.90%, precision of 99.79%, recall of 99.92%, and F1-score of 99.85%. To comprehensively evaluate the proposed framework, extensive experiments are conducted using both a memory-forensics malware dataset and the large-scale BODMAS dataset containing 134,435 PE malware samples spanning 581 malware families. The experimental evaluation incorporates cross-validation, ablation analysis, robustness assessment under multiple perturbation conditions, and feature-space visualization analysis. The results demonstrate that the proposed CAE+LBP framework consistently outperforms standalone feature extraction approaches and conventional end-to-end CNN models while maintaining strong robustness and cross-dataset generalization capability across diverse malware distributions and noisy conditions. **Keywords:** Malware, Visualization, De-noising, Convolutional Neural Network, Autoencoder, Local Binary Pattern, Machine Learning.

Please note: Abbreviations should be introduced at the first mention in the main text – no abbreviations lists. Suggested structure of main text (not enforced) is provided below.

Introduction

Cybersecurity remains one of the most critical defense mechanisms against the continuously evolving landscape of cyber threats targeting organizations, infrastructures, and digital assets¹. However, with the increasing reliance on software applications and the growing sophistication of cyber threats, AI-assisted cybersecurity systems have emerged as a critical enhancement to strengthen organizational defenses by enabling faster, more adaptive, and more effective threat detection and response². Furthermore, the increasing complexity of cyberattacks has led researchers to adopt machine learning (ML) and deep learning (DL) techniques to enhance malware detection and intrusion detection system (IDS) performance³. Although recent research has introduced deep learning-based multi-feature fusion algorithms that often lack adequate interpretability, this limitation is addressed through Convolutional Neural Networks (CNNs) trained on multiple malware executable modalities using visual representations to improve detection performance⁴.

Recent reports continue to demonstrate the rapid growth and sophistication of malware-related attacks. The SonicWall report⁵ revealed a 399% increase in cryptojacking incidents, totaling more than 332 million attacks, alongside a significant rise in encrypted threats. Similarly, cybersecurity reports in 2022 and 2023^{6,7} highlighted substantial increases in ransomware, cloud exploitation, and advanced persistent malware campaigns, emphasizing the urgent need for intelligent and adaptive malware analysis frameworks.

Understanding the behavior and characteristics of malicious software commonly relies on several malware analysis methodologies, including static analysis, dynamic analysis, hybrid analysis, and memory forensics⁸. However, traditional detection methods are limited because they treat physical failures and cyber intrusions separately and rely on unclear models unsuitable for high-stakes decisions⁹. Static analysis investigates malicious files without execution by examining metadata, binary structures, strings, and code patterns^{10,11}. Although efficient, static analysis often struggles against advanced obfuscation, polymorphism, encryption, and previously unseen malware variants¹². Dynamic analysis addresses some of these limitations by executing malware within controlled sandbox environments and monitoring runtime behavior, network activity, system calls, and registry modifications^{13–15}. However, dynamic analysis remains resource-intensive and may fail to capture sophisticated memory-resident or fileless malware^{8,16,17}. Consequently, memory forensics has emerged as an important complementary approach capable of revealing malicious artifacts and hidden runtime behaviors directly from volatile memory^{8,18}.

To reduce reliance on handcrafted forensic expertise, recent malware analysis research has increasingly explored visualization-based approaches that transform malicious binaries, memory dumps, or extracted behavioral artifacts into image representations suitable for machine learning and deep learning analysis^{19–21}. These approaches enable automated extraction of structural and texture-related malware characteristics from grayscale visual representations. Despite promising classification performance reported in prior studies, recent surveys and investigations¹⁷ indicate several persistent limitations within existing visualization-based malware detection frameworks. Most existing approaches rely on relatively limited datasets, insufficient robustness evaluation, and single-dataset experimental validation, while often overlooking the integration of complementary global and local feature representations. Furthermore, visually similar malware families and noise-sensitive samples remain particularly challenging, frequently resulting in degraded classification performance and reduced generalization capability.

Another important limitation of existing malware visualization techniques is the lack of comprehensive evaluation under large-scale and diverse malware distributions. Many existing studies report high classification accuracy using controlled benchmark datasets; however, limited attention has been given to cross-dataset validation, robustness analysis, and scalability assessment under realistic malware diversity conditions. In addition, while several studies employ deep learning models for automatic feature extraction, the complementary integration between latent structural representation learning and local texture-aware descriptors remains underexplored, particularly in memory-forensics-based malware visualization scenarios.

Based on the limitations identified in existing visualization-based malware analysis frameworks, several important research gaps can be observed:

- Most existing visualization-based malware detection methods rely on single-dataset evaluations with limited malware diversity, restricting the assessment of scalability and cross-dataset generalization capability.
- Limited attention has been given to robustness analysis under noisy and perturbed malware visualization conditions, particularly for visually similar and noise-sensitive malware families.
- Existing approaches often rely either on end-to-end supervised deep learning models or handcrafted descriptors independently, while the integration of complementary latent structural representations and local texture-aware features remains underexplored.
- The impact of malware image resolution on classification effectiveness and computational efficiency has not been comprehensively investigated within hybrid malware visualization frameworks.

Accordingly, this study aims to address the following research questions:

- RQ1: Can hybrid integration between CAE-based latent structural learning and LBP-based texture feature extraction improve the classification of visually similar and noise-sensitive malware families?
- RQ2: How robust is the proposed framework under multiple perturbation and noisy visualization conditions?
- RQ3: Can the proposed framework generalize effectively across both memory-forensics malware datasets and large-scale malware distributions containing hundreds of malware families?
- RQ4: What is the impact of malware image resolution on classification effectiveness and computational efficiency within the proposed framework?

Motivated by these limitations, this research proposes a novel hybrid malware representation framework that integrates Convolutional Autoencoder (CAE)-based latent structural learning with Local Binary Pattern (LBP)-based texture feature extraction for malware classification. To the best of our knowledge, this is among the first studies to jointly investigate hybrid latent-texture representation learning within a memory-forensics malware visualization setting while simultaneously evaluating robustness, class separability, and cross-dataset generalization capability. The proposed framework aims to improve the discrimination of visually similar and noise-sensitive malware families by combining global hierarchical representations learned through CAE with fine-grained local texture descriptors extracted using LBP. Unlike many existing visualization-based malware detection approaches that primarily emphasize classification accuracy on constrained benchmark datasets, the proposed framework focuses on constructing stable and generalizable malware representations through the complementary integration of latent structural learning and local texture-aware feature extraction. Furthermore, this study extends beyond conventional single-dataset evaluation by incorporating robustness analysis under perturbation conditions, cross-dataset validation using large-scale malware distributions, feature-space separability analysis, and comprehensive ablation studies. To the best of our knowledge, this work represents one of the first comprehensive investigations of hybrid CAE+LBP representation learning within a memory-forensics malware visualization setting under robustness, scalability, and generalization perspectives.

Unlike conventional end-to-end deep learning approaches that rely solely on discriminative supervised learning, the proposed framework leverages unsupervised representation learning through CAE to capture intrinsic structural patterns and invariant malware characteristics. These representations are further strengthened using LBP descriptors to improve robustness against local perturbations and structural similarity across malware families. In addition, the proposed framework investigates the impact of image resolution on classification effectiveness and computational efficiency by evaluating multiple image dimensions, including 300×300 , 224×224 , and 200×200 .

To comprehensively evaluate the proposed framework, extensive experiments are conducted using both a memory-forensics malware dataset and the large-scale BODMAS dataset containing 134,435 PE malware samples spanning 581 malware families. The experimental evaluation incorporates cross-validation, ablation analysis, baseline comparison against end-to-end CNN models, robustness analysis under multiple perturbation conditions, and feature-space visualization using t-SNE to investigate class separability behavior under challenging malware distributions.

The primary contributions of this study are summarized as follows:

- We propose a novel hybrid malware representation framework that integrates CAE-based latent structural learning with LBP-based texture feature extraction to enhance the classification of visually similar and noise-sensitive malware families.
- We introduce a comprehensive experimental evaluation framework incorporating cross-validation, ablation analysis, robustness assessment under multiple perturbation conditions, and feature-space visualization to rigorously analyze the effectiveness and stability of the proposed approach.
- We conduct extensive validation using both a memory-forensics malware dataset and the large-scale BODMAS dataset containing 581 malware families to evaluate scalability and cross-dataset generalization capability under substantially diverse malware distributions.
- We investigate the impact of malware image resolution on classification effectiveness and computational efficiency by evaluating multiple image dimensions while maintaining lightweight computational complexity.

Related Work and Background

This section reviews existing malware analysis and classification approaches from two primary perspectives: memory-forensics-based malware analysis and visualization-based malware classification techniques employing machine learning and deep learning methodologies.

memory-forensics-based Malware Analysis

Memory forensics has emerged as a promising direction for malware analysis due to its capability to capture volatile runtime artifacts, hidden malicious behaviors, injected processes, API traces, registry modifications, and system-level interactions that are often difficult to identify using conventional static analysis techniques. Consequently, numerous researchers have investigated machine learning-driven memory analysis frameworks for malware detection and classification.

In¹⁰, researchers present a unique approach to malware identification and classification by employing memory forensic techniques to extract memory-based characteristics from memory images. The malware's interactions with the operating system, injection of DLLs and programs, communication with a command and control server, and requests for administrative rights are all revealed by the features extracted. Furthermore, the authors turned the features into binary vectors using a feature

engineering method before training and testing the classifiers. The experimental results show that the proposed method achieves a high classification accuracy of 98.5% when using the SVM classifier, with a false positive rate as low as 1.24%.

In²², researchers present a strategy for detecting malware that is based on memory forensics and incorporates a dynamic analysis by combining information gleaned from harmful artifacts in memory with information gleaned from malware's operation. Prior to applying machine learning models to a data set for training and evaluation, authors used pre-modelling approaches to design features. The experimental findings indicate a considerable improvement in accuracy and false-positive rate.

Beyond conventional memory artifact extraction, several studies explored richer behavioral and structural representations derived from volatile memory analysis. In²³, memory dump features extracted from registry activities, imported libraries, and API-related information were evaluated using machine learning classifiers, where SVM achieved an accuracy of 96%. Similarly, researchers in²⁴ introduced a memory-pattern-based malware detection strategy by analyzing three-dimensional memory access patterns generated during malware execution. Their findings demonstrated that malware samples belonging to the same family exhibit highly similar memory access characteristics, highlighting the forensic value of memory visualization patterns for malware analysis and classification.

Other research efforts focused on virtualization-assisted and cloud-based malware detection frameworks leveraging memory introspection and volatile memory acquisition. In²⁵, researchers introduced a framework for detecting unknown malware within Linux-based cloud environments using 171 features extracted from virtual machine memory regions. Extensive experiments conducted on 21,800 memory dumps demonstrated notable malware detection capability with high true positive rates and low false positive rates. Similarly, the work in²⁶ proposed a secure malware detection framework for virtualized cloud infrastructures using MinHash analysis and transfer learning strategies for identifying unknown malware across different virtual servers while optimizing computational overhead.

Several additional studies further demonstrated the growing importance of memory forensic analysis for cybersecurity and malware investigation. For instance, the framework proposed in²⁷ utilized virtual machine introspection and intelligent decision mechanisms for detecting hidden and suspicious malware processes in real-time environments. In²⁸, researchers introduced the TKRD framework for kernel rootkit detection using cloud-based forensic memory analysis and machine learning classifiers, where random forest achieved the best classification performance. Furthermore, ransomware-oriented forensic analysis approaches such as²⁹ leveraged memory dump artifacts and meta-feature engineering to improve the detection of previously unseen ransomware samples. Collectively, these studies demonstrate the increasing reliance on volatile memory analysis, virtualization-assisted introspection, and machine learning techniques for modern malware detection and forensic investigation.

Visualization-Based Malware Classification Approaches

Alongside memory-forensics-based analysis, visualization-driven malware classification has attracted considerable attention due to its capability to transform malware binaries and behavioral artifacts into image-like representations suitable for machine learning and deep learning analysis. Such approaches reduce dependency on extensive domain-specific feature engineering while enabling automated extraction of discriminative visual patterns from malware samples.

In¹⁸, authors introduce a malware detection method based on VolMemLyzer, a tool designed to extract 36 key features from memory dumps during a live malware infection. To further refine the feature set, the authors employed an additional tree classifier to rank the features, streamlining the selection process. Machine learning classifiers were then used to validate the results, performing both binary and multi-class classification on the reduced feature set.

In²⁷, researchers introduce the A-IntExt system, a next-generation VMI-based in-guest security solution for malware detection. The framework reconstructs executables from digital artifacts in the VM, and machine learning is used to identify malicious files. The authors assert that A-IntExt is highly robust for real-time operations and can be applied to publicly benchmarked datasets with minimal performance overhead.

In²⁸, researchers present a framework called TKRD to detect kernel rootkit-type malware using cloud-based memory forensic analysis. Different machine learning classifiers, including DT, RBC, Bayesian, and SVM, were deployed for training and testing, where RF achieved the best performance with an accuracy of 98.6%.

Similarly, in²⁹, researchers propose a cloud-based ransomware detection method using memory forensic analysis and meta-feature construction derived from memory artifacts extracted using the Volatility framework. The random forest classifier achieved the strongest detection capability among the evaluated classifiers.

Recent advances in malware visualization have increasingly incorporated deep learning and representation learning strategies using CNNs, autoencoders, texture descriptors, and transfer learning architectures. These techniques demonstrated promising classification capability by automatically learning hierarchical malware representations directly from grayscale malware images while reducing the dependency on handcrafted feature engineering. In particular, CNN-based malware visualization frameworks enabled improved spatial feature learning and automated discrimination between malware families exhibiting structurally

similar patterns.

Several studies further explored hybrid and feature-enhanced malware visualization approaches by integrating handcrafted texture descriptors with machine learning or deep learning frameworks. These methods aimed to strengthen local feature extraction capability and improve malware family discrimination through texture-aware representations. In addition, transfer learning architectures such as VGG, ResNet, and autoencoder-based frameworks demonstrated the effectiveness of hierarchical latent representation learning for malware image classification. Nevertheless, many existing visualization-based frameworks primarily focus on improving classification accuracy on relatively constrained benchmark datasets, while limited attention has been devoted to robustness against perturbation conditions, scalability under diverse malware distributions, and cross-dataset generalization capability.

Furthermore, existing approaches often employ either handcrafted feature extraction techniques or end-to-end deep learning architectures independently. The complementary integration between latent structural representation learning and local texture-aware feature extraction remains insufficiently explored, particularly within memory-forensics malware visualization scenarios involving visually similar and noise-sensitive malware families. These limitations motivate the need for more robust hybrid representation learning frameworks capable of improving malware discrimination capability while maintaining stability under noisy and complex malware visualization conditions.

Research Gap and Motivation

Based on conducting a comprehensive literature review, it becomes evident that numerous researchers favor visualization-based methodologies for malware detection, aiming to circumvent the necessity for domain-specific expertise. However, within the realm of multiclassification challenges, the predominant focus among researchers lies in evaluating the holistic performance of sophisticated malware. The lack of investigation in the realm of classification, where numerous classes of advanced malware demonstrate susceptibility to noise and exhibit substantial similarities among them.

Although visualization-based malware analysis has demonstrated promising classification performance, several important limitations remain insufficiently addressed in existing literature. Many existing approaches rely on relatively limited benchmark datasets, lack robustness evaluation under noisy or perturbed conditions, and provide limited analysis of cross-dataset generalization capability. In addition, existing studies often utilize either end-to-end deep learning architectures or handcrafted texture descriptors independently, while the complementary integration between latent structural representation learning and local texture-aware feature extraction remains underexplored, particularly within memory-forensics malware visualization scenarios.

Motivated by these limitations, the proposed framework introduces a hybrid CAE+LBP representation learning strategy that combines hierarchical latent structural learning with fine-grained texture-aware feature extraction to improve malware discrimination capability under visually similar and noise-sensitive conditions. Furthermore, unlike many existing visualization-based malware classification studies that primarily emphasize classification accuracy alone, the proposed work jointly investigates robustness under perturbation conditions, cross-dataset scalability, feature-space separability, and generalization capability within a unified malware visualization framework.

To fill this gap, our research endeavors to implement a hybrid feature extraction approach. This approach not only aims to improve the overall efficacy of malware detection but also specifically enhances the detection performance of malware classes characterized by high similarity and susceptibility to noise.

Therefore, the proposed work differs from existing visualization-based malware classification studies by jointly investigating hybrid latent-texture representation learning, robustness under perturbation conditions, cross-dataset scalability, and feature-space separability within a unified memory-forensics malware analysis framework.

Methodology

The proposed methodology is designed for the effective detection and classification of advanced malware variants. The overall pipeline consists of four main stages: (i) malware execution and memory acquisition, (ii) image generation and preprocessing, (iii) hybrid feature extraction using CAE and LBP, and (iv) classification using machine learning models. In this study, memory dump files generated from a virtual machine (VM) after executing both malicious and benign samples are utilized. Let $M = \{m_1, m_2, \dots, m_n\}$ denote the set of memory dump files, where each m_i corresponds to an executed sample. This enables a detailed examination of the underlying patterns and distinctive features present in memory representations.

To enhance the robustness of feature extraction, the CAE is employed for its capability to capture deep hierarchical and spatial relationships within malware images, effectively learning complex structural dependencies. Formally, the CAE learns a mapping $f_\theta : X \rightarrow Z$, where X represents the input image space and Z denotes the latent feature-space. The encoder compresses the input into a lower-dimensional representation, while the decoder reconstructs the input to preserve essential structural information. In conjunction, the LBP technique is integrated to extract fine-grained texture characteristics that complement the CAE by emphasizing local pixel-level intensity variations often indicative of specific malware families. The LBP operator transforms each pixel by comparing it with its neighborhood, generating a texture descriptor that encodes local structural patterns.

Specifically, the proposed framework employs an early fusion strategy, where the latent feature vector extracted from the

CAE encoder, denoted as $Z \in \mathbb{R}^d$, is concatenated with the LBP histogram feature vector $L \in \mathbb{R}^k$ to form a unified representation $F \in \mathbb{R}^{d+k}$. The fused feature vector is defined as $F = [Z || L]$, where $||$ denotes vector concatenation.

In this configuration, the CAE latent representation captures high-level structural and spatial information, while the LBP histogram encodes local texture patterns. Prior to fusion, both feature vectors are normalized to ensure consistent scaling. The resulting fused representation enables the classifier to learn complementary information from both global and local feature-spaces.

In our implementation, the CAE encoder produces a latent feature vector of dimension $d = 512$, while the LBP histogram yields a feature vector of dimension $k = 256$. Consequently, the fused feature vector has a total dimensionality of $d + k = 768$. Early fusion is adopted to allow the classifier to learn joint representations from both global and local features, enhancing discriminative capability.

Initial experimentation involves the execution of advanced malware within a virtual environment, followed by the capture of VM memory, which is subsequently transformed into visual representations. The malware samples are executed in an isolated virtual machine environment to ensure safe and controlled behavior monitoring. After execution, the memory state of the VM is captured as a raw memory dump file. Each memory dump is then processed by reading its byte sequence and mapping it directly to pixel intensities to generate a grayscale image representation.

These images, with pixel values ranging from 0 to 255, are resized to fixed dimensions, including 300×300 , 224×224 , and 200×200 , to determine the optimal image size for grayscale-based analysis. Formally, each memory dump m_i is converted into an image $I_i \in \mathbb{R}^{h \times w}$, where h and w correspond to the selected image dimensions. The resizing operation is performed using bilinear interpolation to ensure consistency across all samples.

The visual malware images are then fed into a hybrid approach for feature extraction, comprising two essential components: (i) deep hierarchical feature extraction using a Convolutional Autoencoder (CAE), and (ii) local texture feature extraction using the Local Binary Pattern (LBP) operator. The CAE processes the input image to produce a latent representation capturing global structural dependencies, while the LBP operator computes local texture descriptors based on neighborhood intensity comparisons.

The extracted features are subsequently utilized in machine learning classifiers for the purpose of malware detection and classification. Specifically, the fused feature vector F is provided as input to the classifier, where XGBoost is employed as the primary model due to its effectiveness in handling high-dimensional feature-spaces and non-linear decision boundaries.

Figure 1 provides a detailed overview of our research methodology. The figure illustrates the complete pipeline, including data acquisition, image generation, feature extraction, feature fusion, and classification stages.

The methodology consists of three primary steps, each detailed as follows:

A. Step 1

In the initial step of the methodology, the transformation of malware files from binary format into a visual representation is explained. This process follows a structured pipeline consisting of data acquisition, controlled execution, memory extraction, and image generation. This intricate process encompasses several sub-steps crucial to the conversion.

Firstly, datasets pertinent to the malware specimens are gathered and organized for subsequent analysis. The dataset comprises both malicious and benign samples obtained from publicly available malware repositories. Each sample is uniquely identified and stored for controlled execution.

Following this, the technique of memory dumping is employed to extract key information and patterns from the binary files. Each sample is executed within an isolated VirtualBox environment to ensure safe and controlled analysis. After execution, a full memory snapshot of the virtual machine is captured using a memory acquisition tool, generating a raw memory dump file corresponding to each executed sample.

Subsequently, these extracted data elements are channelled into a visualization format, allowing for a comprehensible and insightful representation of the malware characteristics. Specifically, each memory dump file is processed as a sequence of bytes, where each byte value (ranging from 0 to 255) is directly mapped to a pixel intensity in a grayscale image. The byte stream is reshaped into a two-dimensional matrix to form an image, which is then resized to predefined dimensions (300×300 , 224×224 , and 200×200) using bilinear interpolation. This transformation ensures consistency across samples and enables the application of image-based feature extraction techniques.

1. Dataset and memory dumping

In the field of malware detection and classification, four widely used datasets are often employed to identify and categorize malware: BIG2015³⁰, Mallmg³¹, Malicia³², and Malevis³². These datasets are primarily based on static analysis and image-based representations; however, they do not provide the corresponding raw Portable Executable (PE) files or memory-level artifacts required for dynamic memory-based analysis.

However, it is important to highlight that none of these datasets contain the PE (Portable Executable) files required for analyzing the source files directly. As a result, they are not suitable for studies that rely on memory forensics and runtime

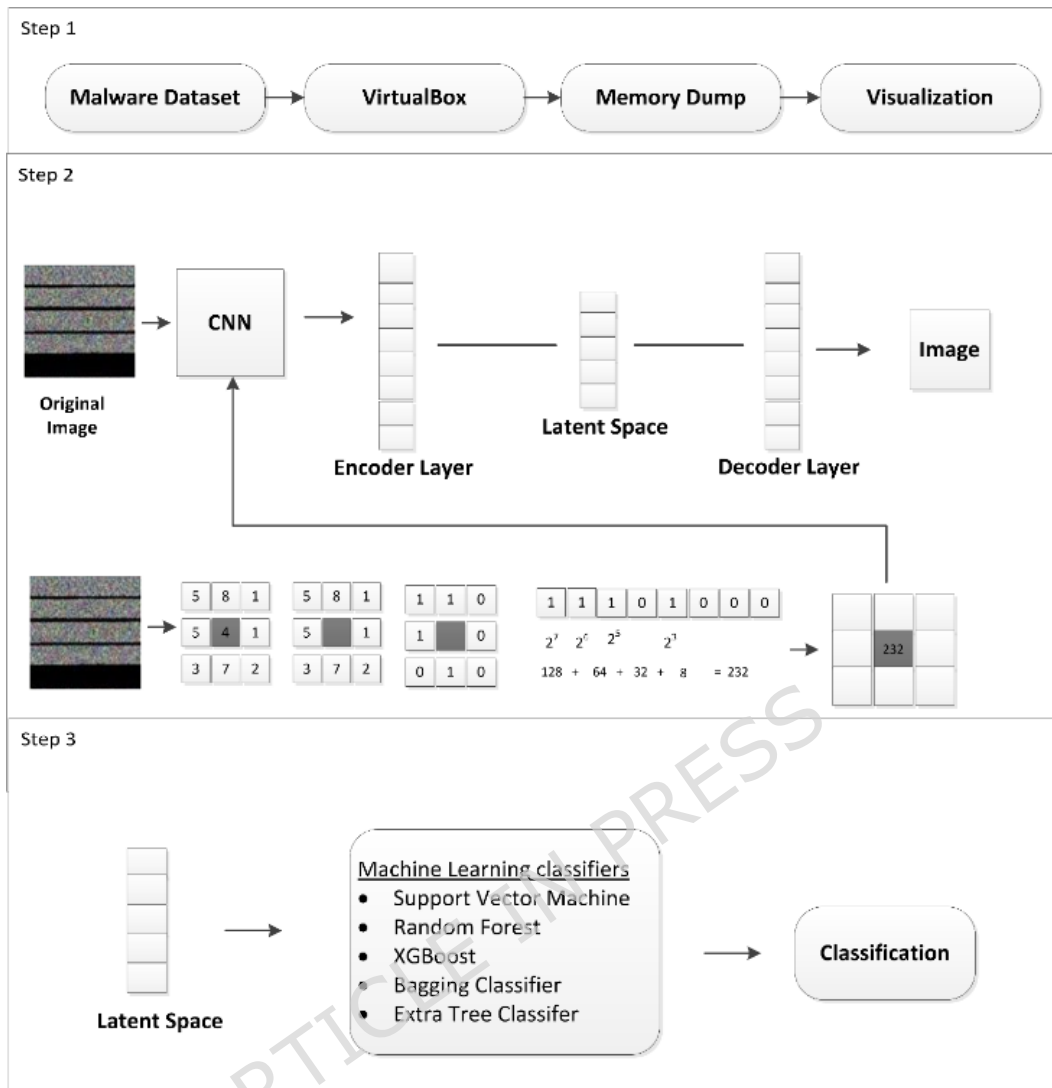


Figure 1. Overview of the proposed malware classification pipeline. Step 1: malware samples are executed in a virtual machine and memory dumps are converted into grayscale images. Step 2: hybrid feature extraction is performed using CAE (for latent representations) and LBP (for texture features). Step 3: the fused features are classified using machine learning models such as SVM, Random Forest, and XGBoost.

behavior extraction. To bridge this gap, a memory-forensics-based malware dataset³³ was chosen for analysis.

The selected dataset consists of 4,294 memory dump files, including both malicious and benign samples. Each dump file corresponds to the memory state of a system after executing a specific sample within a controlled virtual environment. The dump files are generated by setting up a virtual Windows 10 environment, where the system's memory image is captured using Procdump and then saved as dump files for further examination.

To ensure reproducibility, the dataset can be reconstructed by executing malware and benign samples within a Windows 10 virtual machine (VirtualBox) environment, followed by capturing the process memory using Procdump with default settings. Each execution produces a dump file representing the runtime memory state, which is subsequently used for visualization and feature extraction. The use of a controlled virtual environment ensures consistency across samples while preventing unintended system-level interference.

2. Visualization

To facilitate the conversion from binary data to grayscale images, we employ a Python script. The visualization process is implemented using a deterministic transformation pipeline to ensure reproducibility across all samples. This conversion process involves two significant stages: (i) converting binary data to 8-bit format and (ii) presenting the 8-bit data as an image.

In the first stage, each memory dump file is read as a sequence of raw bytes. Let $m_i = \{b_1, b_2, \dots, b_k\}$ denote the byte sequence of the i -th dump file, where each $b_j \in [0, 255]$. Since each byte already represents an 8-bit value, no additional encoding is required. The byte sequence is directly interpreted as pixel intensity values.

The term “8-bit” pertains to a technique for encoding graphic data in a computer’s memory or within image files. In this context, each byte is mapped to a grayscale pixel intensity, where a value of 0 corresponds to black and 255 corresponds to white. The primary objective is to represent each pixel using 8 bits, equivalent to 1 byte.

Images encoded in this manner can be rendered in both color and grayscale. In grayscale images, only a single channel is present, while color images encompass three channels: R (red), G (green), and B (blue). In this study, grayscale representation is selected to reduce computational complexity and focus on structural and texture-based patterns relevant to malware behavior.

The pixel values in these images fall within the range of 0 to 255. This range is calculated using the formula $2^n - 1$, where n signifies the number of bits used. In the second stage, the byte sequence is reshaped into a two-dimensional matrix to form an image. Given a target image size ($h \times w$), the first $h \times w$ bytes are used, and if the sequence length is insufficient, zero-padding is applied. Conversely, if the sequence is longer, excess bytes are truncated to maintain a fixed-size representation.

The resulting matrix is then normalized and resized to predefined dimensions (300×300 , 224×224 , and 200×200) using bilinear interpolation. This ensures uniformity across all samples and enables consistent input to the feature extraction stage.

The pseudocode of transforming the binary files into visualization is represented in Algorithm 1, which provides a step-by-step procedure for converting raw memory dump files into grayscale image representations.

Algorithm 1: Binary-to-Image Conversion Process

```

procedure GET_BINARY_DATA(filename)
  binary_values  $\leftarrow$  []
  Open file with filename in binary mode
  while not end of file do
    Read one byte  $b$ 
    Append integer value of  $b$  to binary_values
  return binary_values

procedure GENERATE_GRAYSCALE_IMAGE(filename, width, height)
  binary_data  $\leftarrow$  GET_BINARY_DATA(filename)
   $N \leftarrow width \times height$ 
  if length(binary_data) <  $N$  then
    Pad binary_data with zeros until length =  $N$ 
  else
    Truncate binary_data to first  $N$  elements
  Reshape binary_data into matrix  $I \in \mathbb{R}^{height \times width}$ 
  Normalize values to range [0,255] if required
  Create grayscale image using matrix  $I$ 
  Save image with same filename and extension .png

procedure MAIN(input_dir, width, height)
  for each file  $f$  in input_dir do
    GENERATE_GRAYSCALE_IMAGE( $f$ , width, height)

```

B. Step 2

In the realm of malware detection and classification, a crucial step in both machine learning and deep learning models is the process of feature engineering. Many researchers have applied various feature engineering techniques, including GISTs³⁴, HOG³⁵, PCA³⁶, and Wavelet Transform^{8,37}. However, it has been observed that these techniques exhibit sensitivity to specific malware classes, particularly those that are susceptible to similarities and noise, leading to a reduction in overall performance. In this work, our proposal centers on the utilization of a Convolutional Autoencoder (CAE) for the extraction of robust and discriminative features from malware images. Figure 3 illustrates the proposed CAE architecture employed for latent feature extraction from malware images. The framework is designed to learn compact hierarchical representations that preserve the global structural characteristics of malware visualization patterns while reducing redundancy and sensitivity to noisy perturbations before feature fusion with LBP descriptors.

Unlike conventional feature extraction techniques, the CAE is designed to learn hierarchical and spatial representations directly from the input images. Given an input image $I \in \mathbb{R}^{h \times w}$, the CAE learns a mapping $f_\theta : I \rightarrow Z$, where $Z \in \mathbb{R}^d$ represents

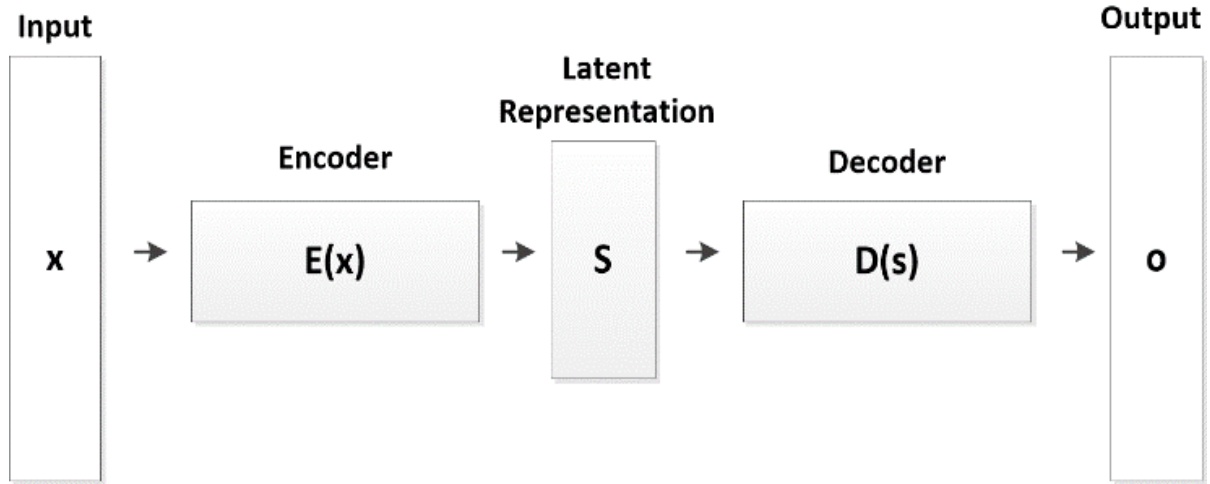


Figure 2. Illustration of the autoencoder framework. The encoder $E(x)$ maps the input image x into a latent representation s , which captures the most informative features of the data. The decoder $D(s)$ reconstructs the input from this latent space. In this study, this framework is implemented as a Convolutional Autoencoder (CAE) to extract compact and discriminative feature representations from malware images.

the latent feature vector capturing the most informative characteristics of the malware sample.

An autoencoder is an artificial neural network designed for unsupervised learning applications, which do not rely on class labels or labelled data. In this study, a convolutional variant of the autoencoder is employed to better exploit spatial correlations in malware images. Its primary objectives include reducing dimensionality, extracting features, and compressing data.

The encoder's function is to compress the input data by reducing its dimensionality, resulting in a more compact representation known as the latent space or code. If the input data is represented as x and the encoder as E , the latent space representation s can be expressed as $s = E(x)$. Similarly, the decoder reconstructs the input as $o = D(s)$, resulting in $o = D(E(x))$.

The CAE architecture used in this study consists of three convolutional layers in the encoder with filter sizes of 32, 64, and 128, respectively, each followed by ReLU activation and max-pooling layers to progressively reduce spatial dimensions. The decoder mirrors this structure using upsampling layers to reconstruct the input image. The latent space dimension is fixed to $d = 128$, representing the compressed feature vector extracted from each input image.

The model is trained using the Adam optimizer with a learning rate of 1×10^{-4} , a batch size of 32, and a maximum of 50 epochs. Mean Squared Error (MSE) is used as the reconstruction loss function, defined as $\mathcal{L} = \|I - \hat{I}\|^2$, where $\hat{I}$ denotes the reconstructed image. Early stopping with a patience of 10 epochs is applied to prevent overfitting.

After training, the decoder is discarded, and only the encoder component is used to extract the latent representation Z for each input image. These latent vectors serve as global feature descriptors and are subsequently combined with LBP features in the hybrid feature extraction stage.

Figure 2 offers an elaborate visualization of the standard autoencoder architecture. In this work, the illustrated structure is adapted to a convolutional framework to better capture spatial dependencies in malware images.

As illustrated in Fig. 1, the proposed methodology employs a hybrid feature engineering approach that integrates a Convolutional Autoencoder (CAE) and Local Binary Pattern (LBP) for effective malware classification. In our proposed methodology, we investigate the hybrid feature engineering approach that comprises a CAE and LBP to extract features for the purpose of malware classification.

A CAE is an unsupervised learning algorithm that involves the design of a neural network (NN) capable of both encoding and decoding data, with the aim of reconstructing the input. In this study, the CAE is specifically utilized as a feature extractor rather than solely for reconstruction, where the encoder learns compact representations of malware images that preserve discriminative structural information. CAEs have been shown to be effective in compressing spatial field data, as demonstrated in prior research³⁸.

Notably, there are convolutional models that can manage both spatial and temporal dimensions. A CAE combines local convolutional operations with the autoencoder structure, where the process involves adding a reconstruction objective to the convolutional transformation. Formally, given an input image x , the encoder applies a series of convolutional operations to generate a latent representation $s = E(x)$, while the decoder reconstructs the input as $o = D(s)$. The training objective is to minimize the reconstruction error between x and o .

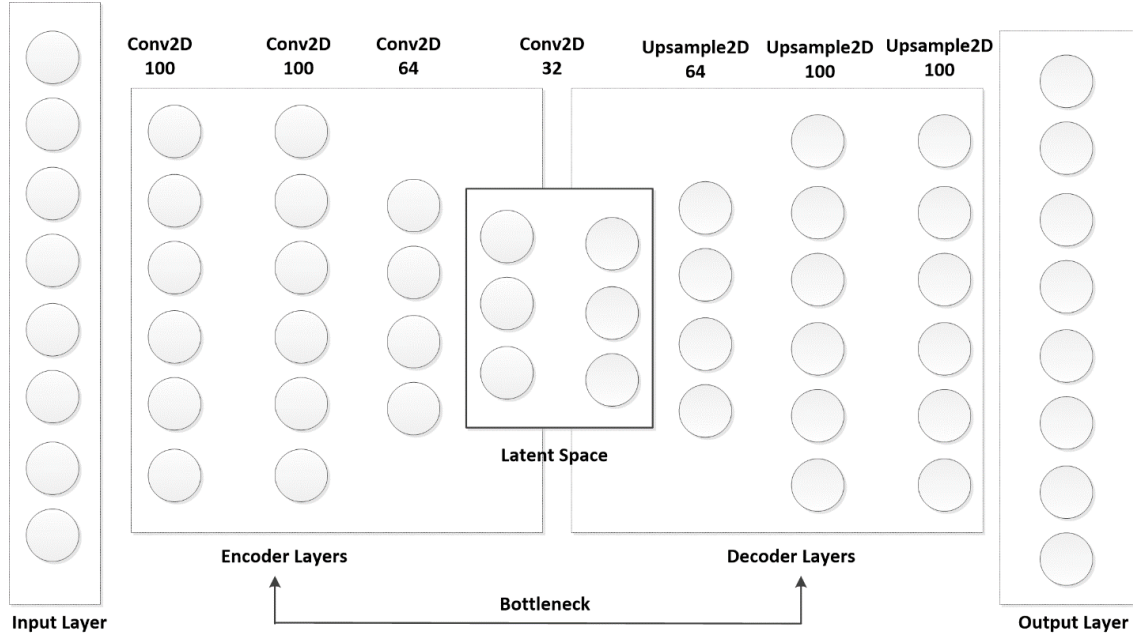


Figure 3. Proposed CAE-based latent feature extraction architecture for malware image representation learning. The framework encodes malware visualization inputs into compact hierarchical latent representations that preserve global structural characteristics while reducing redundancy and noise sensitivity prior to hybrid feature fusion with LBP descriptors.

The transformation of input feature maps into the latent space is known as the convolutional encoder, while the output values are reconstructed via the inverse convolution operation, referred to as the convolutional decoder. In the proposed framework, only the encoder component is retained after training to extract latent features, while the decoder is discarded. Furthermore, by utilizing standard autoencoder unsupervised training, the parameters for both the encoding and decoding operations can be optimized and learned.

The operation within each CAE layer can be formally described as follows. The operation within the CAE layer, as illustrated in Figure 3, begins with the input feature maps denoted as $x \in \mathbb{R}^{n \times l \times l}$, where n represents the number of feature maps, and $l \times l$ corresponds to the spatial dimensions of each map. These input feature maps can be derived from either the initial input layer or the previous layer in the network.

It comprises n feature maps, with each feature map having a size of $l \times l$ pixels. The CAE operation uses m convolutional kernels, producing an output layer with m feature maps. Each output feature map is computed by convolving the input feature maps with learnable kernels, followed by a non-linear activation function (ReLU in this study). When the input feature maps are generated from the input layer, n represents the number of input channels. However, if the input feature maps come from the previous layer, n refers to the number of output feature maps generated by that preceding layer.

The convolution operation for the k -th output feature map can be expressed as:

$$h_k = \sigma \left(\sum_{i=1}^n x_i * W_{k,i} + b_k \right)$$

where $*$ denotes convolution, $W_{k,i}$ represents the convolution kernel connecting the i -th input feature map to the k -th output feature map, b_k is the bias term, and $\sigma(\cdot)$ denotes the activation function.

This distinction reflects how the number of feature maps evolves depending on the stage of the network. The dimensions of the convolutional kernel are represented as $d \times d$, with the condition that $d \leq l$. In this study, fixed kernel sizes and pooling operations are used to progressively reduce spatial dimensions while preserving critical structural features.

The learnable parameters of the CAE are denoted as $\theta = (W, W', b, b')$, where W and b correspond to the encoder parameters, and W' and b' represent the decoder parameters.

In the encoder, the convolutional kernels are defined as $W = \{W_k\}_{k=1}^m$, where each kernel $W_k \in \mathbb{R}^{n \times d \times d}$ operates across n input feature maps with spatial size $d \times d$. The corresponding bias terms are represented as $b = \{b_k\}_{k=1}^m$, where each $b_k \in \mathbb{R}$.

Similarly, in the decoder, the parameters are denoted as $W' = \{W'_k\}_{k=1}^n$ and $b' = \{b'_k\}_{k=1}^n$, where W'_k represents the deconvolution (or transposed convolution) kernels used to reconstruct the input feature maps from the latent representation.

Unlike fully connected autoencoders, the CAE preserves the spatial structure of the input data through convolutional operations, eliminating the need for explicit vectorization of feature maps. This design enables efficient parameter sharing and improves the model's ability to capture spatial dependencies within malware images.

Initially, the input image is processed through the encoding stage, where convolutional operations are applied over the entire spatial domain of the image. Instead of explicitly selecting individual patches, the convolutional kernel slides across the input feature maps to extract local patterns.

Let $x \in \mathbb{R}^{n \times h \times w}$ denote the input feature maps, and W_k represent the k -th convolutional kernel. The output feature map corresponding to the k -th kernel is computed as:

$$h_k = \sigma(x * W_k + b_k) \quad (1)$$

where $*$ denotes the convolution operation, b_k is the bias term, and $\sigma(\cdot)$ is the activation function. This operation is applied across all spatial locations to produce the encoded feature maps.

Here, in Eq. (1) we employ a non-linear activation function (σ), specifically the Rectified Linear Unit (ReLU), in our proposed CAE technique.

The ReLU function is defined as:

$$f(x) = \max(0, x) \quad (2)$$

ReLU is selected due to its computational efficiency and its ability to mitigate the vanishing gradient problem, thereby enabling faster convergence during training. It introduces non-linearity while preserving positive feature responses, which is particularly effective for learning hierarchical representations in convolutional architectures.

Although various activation functions such as Sigmoid can be used in neural networks, in this study the Rectified Linear Unit (ReLU) is consistently adopted due to its efficiency and effectiveness in training deep convolutional models. Therefore, Sigmoid activation is not utilized in the proposed CAE architecture.

Following the encoding stage, the decoder reconstructs the input image from the latent representation. Given the encoded feature maps h_k , the reconstruction process is defined as:

$$\hat{x} = D(h) = \sigma\left(\sum_k h_k * W'_k + b'\right) \quad (3)$$

where W'_k denotes the transposed convolution (deconvolution) kernels, b' represents the bias term, and $\hat{x}$ is the reconstructed image. The decoder mirrors the encoder structure to recover spatial information from the latent space.

The training objective of the CAE is to minimize the reconstruction error between the original input image x and the reconstructed image $\hat{x}$. In this work, Mean Squared Error (MSE) is employed as the loss function, defined as:

$$\mathcal{L}_{MSE} = \frac{1}{N} \sum_{i=1}^N \|x_i - \hat{x}_i\|^2 \quad (4)$$

where N denotes the number of training samples. This loss function encourages the model to preserve the structural characteristics of malware images during reconstruction, which is essential for learning meaningful latent representations.

The algorithm for performing feature engineering on visual malware images using CAE is presented in Algorithm 2.

Algorithm 2: Convolutional Autoencoder (CAE) for Feature Extraction

Input: Input image shape $(h, w, 1)$
Output: Trained encoder model for feature extraction

```

// Encoder
input_img ← Input(shape=(h, w, 1))
x ← Conv2D(100, (3,3), activation='relu', padding='same')(input_img)
x ← MaxPooling2D((2,2), padding='same')(x)
x ← Conv2D(100, (3,3), activation='relu', padding='same')(x)
x ← MaxPooling2D((2,2), padding='same')(x)
x ← Conv2D(64, (3,3), activation='relu', padding='same')(x)
x ← MaxPooling2D((2,2), padding='same')(x)
x ← Conv2D(32, (3,3), activation='relu', padding='same')(x)
encoded ← MaxPooling2D((2,2), padding='same')(x)
// Decoder
x ← UpSampling2D((2,2))(encoded)
x ← Conv2D(32, (3,3), activation='relu', padding='same')(x)
x ← UpSampling2D((2,2))(x)
x ← Conv2D(64, (3,3), activation='relu', padding='same')(x)
x ← UpSampling2D((2,2))(x)
x ← Conv2D(100, (3,3), activation='relu', padding='same')(x)
x ← UpSampling2D((2,2))(x)
decoded ← Conv2D(1, (3,3), activation='sigmoid', padding='same')(x)
// Model construction
autoencoder ← Model(input_img, decoded)
// Training configuration
Compile autoencoder with: optimizer = Adam (learning rate =  $1 \times 10^{-4}$ )
loss = Mean Squared Error (MSE)
Train autoencoder for 50 epochs with batch size = 32 using early stopping (patience = 10)
// Feature extraction encoder ← Model(input_img, encoded)
return encoder

```

Unlike supervised CNN models that learn task-specific discriminative features directly optimized for classification, the CAE is trained in an unsupervised manner to reconstruct the input data. This enables the CAE to capture intrinsic structural patterns and underlying data distributions without bias toward specific class labels. As a result, the learned latent representation preserves global spatial dependencies and invariant characteristics that are less sensitive to noise and intra-class variations.

This property is particularly important in malware image analysis, where visually similar classes may share overlapping discriminative features. By learning a compact and generalizable representation, the CAE provides complementary information to the LBP, which focuses on local texture descriptors. The combination of CAE and LBP therefore integrates global structural understanding with fine-grained local variations, enhancing the overall discriminative capability of the feature-space.

In contrast, end-to-end supervised CNN models may overfit to dataset-specific patterns and can be more sensitive to noise or minor perturbations, as their feature representations are directly optimized for classification objectives. The proposed hybrid approach mitigates this limitation by leveraging unsupervised representation learning alongside robust texture descriptors.

Similarly, the LBP algorithm constitutes a pivotal technique within computer vision, aimed at the extraction of salient image features. In this study, LBP is employed to capture fine-grained local texture patterns from malware images, complementing the global representations learned by the CAE. Its extensive utilization spans across diverse applications within the domain of computer vision, encompassing areas such as facial recognition, object detection, and texture analysis.

LBP functions by capturing the spatial correlation between the central pixel and its neighboring pixels by evaluating their intensity differences and generating a binary representation. This binary output is set to one when the intensity of the neighbor equals or exceeds that of the central pixel, and zero otherwise.

The mathematical formulation of LBP is defined in Eq. (5),

$$\text{LBP}_{P,R}(x_c, y_c) = \sum_{p=0}^{P-1} s(I(x_p, y_p) - I(x_c, y_c)) \times 2^p \quad (5)$$

where (x_c, y_c) denotes the coordinates of the central pixel, (x_p, y_p) represents the coordinates of the P neighboring pixels located on a circle of radius R , and $s(\cdot)$ is a thresholding function defined as:

$$s(x) = \begin{cases} 1, & \text{if } x \geq 0 \\ 0, & \text{otherwise} \end{cases}$$

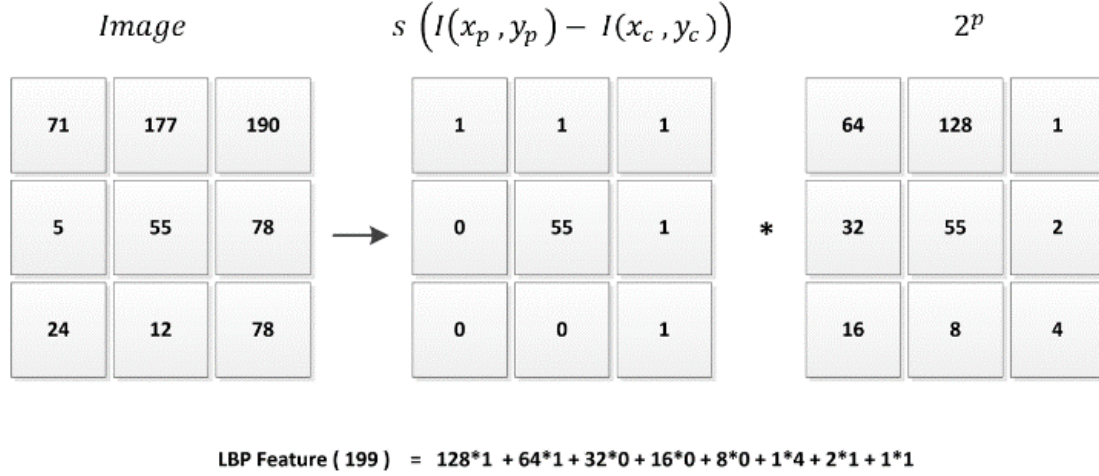


Figure 4. Illustration of the Local Binary Pattern (LBP) computation. A 3×3 neighborhood (corresponding to $P = 8$ and $R = 1$) is used, where each neighboring pixel is compared with the central pixel to generate a binary pattern. The resulting binary values are weighted by powers of two and summed to produce the final LBP code, which serves as a local texture descriptor.

In this work, the parameters are set to $P = 8$ and $R = 1$, which correspond to a standard neighborhood configuration widely used in texture analysis.

After computing the LBP code for each pixel, a histogram of LBP values is generated over the entire image, resulting in a feature vector $L \in \mathbb{R}^k$, where k denotes the number of possible LBP patterns. This histogram representation captures the distribution of local texture patterns and serves as the LBP feature descriptor for each malware image.

In the LBP formulation, P denotes the number of neighboring pixels considered in the local neighborhood, while R represents the radius defining the distance between the central pixel and its neighbors. The intensity of the central pixel is given by $I(x_c, y_c)$, and the intensities of the neighboring pixels are represented as $I(x_p, y_p)$. The thresholding function $s(\cdot)$ outputs 1 when the difference between the neighboring and central pixel intensities is non-negative, and 0 otherwise.

Figure 4 illustrates the LBP computation process for a grayscale image using a standard 3×3 neighborhood configuration (i.e., $P = 8$ and $R = 1$). This configuration enables the extraction of local texture patterns, which are subsequently aggregated into histogram-based feature representations for classification.

The proposed Local Binary Pattern (LBP) feature extraction process is summarized in Algorithm 3, which computes local texture descriptors based on intensity relationships between each pixel and its neighboring pixels. The procedure consists of the following steps:

- **Image Traversal:** The algorithm iterates over all pixels in the image domain, excluding border pixels, to ensure valid neighborhood computation.
- **Neighborhood Definition:** For each pixel located at (x_c, y_c) , a 3×3 neighborhood (corresponding to $P = 8$ and $R = 1$) is considered.
- **Intensity Comparison:** The intensity of each neighboring pixel $I(x_p, y_p)$ is compared with the central pixel intensity $I(x_c, y_c)$.
- **Binary Encoding:** A binary value is assigned based on the comparison, where 1 indicates $I(x_p, y_p) \geq I(x_c, y_c)$, and 0 otherwise.
- **LBP Code Generation:** The binary sequence is converted into a decimal value using a weighted summation based on powers of two, as defined in Eq. (5). This value represents the LBP code for the given pixel.
- **Feature Aggregation:** The computed LBP codes for all pixels are aggregated into a histogram representation, forming the final LBP feature vector for the image.

To further evaluate the robustness of the proposed feature extraction framework, we consider its behavior under controlled noise perturbations. The combination of CAE-based latent representations and LBP texture descriptors is expected to enhance

Algorithm 3: Local Binary Pattern (LBP) Feature Extraction

```

Function compute_lbp_pixel(img, x, y)
  center  $\leftarrow$  img[x, y]
  values  $\leftarrow$  []
  for  $i = -1$  to  $1$  do
    for  $j = -1$  to  $1$  do
      if  $i = 0$  and  $j = 0$  then
         $\perp$  continue
      if  $\text{img}[x + i, y + j] \geq \text{center}$  then
         $\perp$  values.append(1)
      else
         $\perp$  values.append(0)
  result  $\leftarrow$  0
  for  $k = 0$  to  $\text{length}(\text{values}) - 1$  do
     $\perp$  result  $\leftarrow$  result + values[k]  $\times 2^k$ 
  return result

Function compute_lbp_features(img)
  rows, cols  $\leftarrow$  img.shape
  lbp_map  $\leftarrow$  empty matrix of size (rows - 2, cols - 2)
  for  $i = 1$  to rows - 2 do
    for  $j = 1$  to cols - 2 do
       $\perp$  lbp_value  $\leftarrow$  compute_lbp_pixel(img, i, j)
       $\perp$  lbp_map[i-1, j-1]  $\leftarrow$  lbp_value
  // Histogram generation (feature vector) histogram  $\leftarrow$  compute_histogram(lbp_map)
  Normalize histogram
  return histogram

```

stability against noise, as CAE captures global structural patterns while LBP encodes local invariant features. A detailed robustness analysis under different noise conditions is presented in the Experimental Results section.

C. Step 3

In this stage, the extracted feature representations are utilized for classification. Specifically, the hybrid feature vector $F = [Z || L]$, obtained by combining the CAE latent features Z and the LBP texture features L , is used as the input to machine learning classifiers.

We have implemented several well-known classifiers, including Support Vector Machine (SVM) with a Radial Basis Function (RBF) kernel, Random Forest, XGBoost (Extreme Gradient Boosting), Bagging Classifier, and Extra Trees. For SVM, the RBF kernel is used with a regularization parameter $C = 1.0$ and kernel coefficient $\gamma = \text{scale}$. Random Forest and Extra Trees are configured with 100 estimators. XGBoost is implemented with 100 estimators, a maximum tree depth of 6, and a learning rate of 0.1.

All classifiers are trained using the same dataset split, where 80% of the data is used for training and 20% for testing. To ensure robustness, experiments are conducted with consistent preprocessing and feature extraction pipelines across all models.

To further enhance the statistical reliability of the evaluation, we additionally employ k -fold cross-validation with $k = 5$. The dataset is partitioned into five equal subsets, where in each iteration, four subsets are used for training and one subset is used for testing. This process is repeated five times such that each subset is used exactly once as the test set. The final performance is reported as the mean and standard deviation across all folds.

We assess the performance of the machine learning classifiers by considering metrics such as accuracy, precision, recall, and the F1-score. These metrics provide a comprehensive evaluation of classification performance, particularly in handling class imbalance and distinguishing between similar malware families.

Experimental Results

The experimental procedure commences with the exploration of the most suitable size for visual malware images. The greater the dimensionality of these images, the higher the computational and memory resources required. Calculating the time and

space complexity of the image dimensionality using the proposed LBP pseudocode as defined earlier, involves considering the number of operations performed and the memory required concerning the image size.

- **Iterating Through Image Pixels:** The pseudocode iterates through each pixel of the image, excluding the border pixels leading to a loop execution of $O(M \times N)$ where M and N represents the dimensions of the image.
- **Neighborhood Comparison:** Within each pixel iteration, there are two nested loops (3×3 neighborhood), resulting in a constant time complexity $O(1)$ for neighborhood comparisons.
- **Calculating LBP values:** The operations involved in comparing pixel intensities and forming the binary pattern take constant time for each pixel, resulting in a time complexity of $O(1)$ per pixels.

Hence, the overall time complexity is $O(M \times N)$ where M and N are dimensions of the image. Similarly, to calculate the space complexity of the malware visual image dimensionality with respect to the LBP pseudocode,

- **LBP Features Storage:** The accumulates LBP values in the 'lbp_features' list, which potentially holds $M \times N$ values. Therefore, the space complexity for storing the LBP features is $O(M \times N)$.
- **Variables:** The code uses few additional variables like 'center', 'values', and 'results' which occupy constant space regardless of image size. Hence, the space complexity for these variables is $O(1)$.

Considering both aspects, the overall space complexity of the code is $O(M \times N)$ due to the storage requirement for LBP features.

The approach taken by the author in reference³⁴, who examined multiple image sizes and achieved superior accuracy using a 4096-pixel size of visual malware images that, however, demanded extensive computational resources for training, testing, and memory storage. In light of these limitations, our study addresses this challenge by evaluating three distinct image sizes for visual malware analysis: 300×300 , 224×224 , 200×200 . Following the methodology employed in reference⁸, which involved the application of denoising techniques to eliminate noise from visual malware images, it is worth noting that the authors exclusively focused on image sizes of 224×224 pixels, without delving into an exploration of alternative dimensionality options.

The selected image resolutions (300×300 , 224×224 , and 200×200) were chosen to balance classification performance and computational efficiency. Preliminary observations indicated that resolutions lower than 200×200 resulted in excessive loss of structural and texture-related information, particularly for visually similar malware families. Since the proposed framework relies on both global structural representations extracted by the CAE and local texture descriptors captured by LBP, preserving sufficient spatial detail is essential for reliable feature extraction. Consequently, 200×200 was considered the minimum resolution capable of maintaining discriminative visual characteristics while still reducing computational complexity compared to larger image sizes.

This research encompasses two distinct types of experimentation. Firstly, we implemented denoising techniques in combination with machine learning. Secondly, we employed our proposed Hybrid techniques in conjunction with machine learning across various sets of visual malware images. In the final phase of our study, we conducted a comparative analysis of the outcomes obtained from the denoising approach and the hybrid approach. This comparison aimed to assess the performance of each malware class, evaluating key metrics such as accuracy, precision, recall, and the F1-score.

The robustness of the proposed model is an inherent property of the hybrid CAE+LBP design. The process begins with the CAE, which, during its bottleneck compression, discards high-frequency noise. The resulting cleaned representation is then processed by the LBP operator, which calculates texture descriptors that are invariant to monotonic grey-scale changes and minor structural distortions. The synergy of this two-stage process of noise suppression followed by invariant feature extraction serves as an effective regularizer, diminishing overfitting and improving generalization without necessitating computationally intensive techniques like extensive augmentation.

Experimental Setup

This subsection provides a comprehensive description of the experimental setup, including the computational environment, data preparation process, feature extraction pipeline, and evaluation protocol. Table 1 presents the hardware specifications used in the experiments.

The implementation is carried out using Python, supported by essential libraries including OpenCV (CV2) for image processing, Matplotlib for visualization, Pandas for data handling, and Scikit-image for feature extraction. Experiments were conducted using Python 3.14 with libraries including NumPy, Scikit-learn, TensorFlow/Keras, and OpenCV.

The experimental pipeline follows a structured workflow to ensure consistency and reproducibility. The dataset consists of 4,294 memory dump files representing both malicious and benign samples, obtained from a publicly available memory

Table 1. Hardware configuration used for training and evaluating the proposed malware classification models.

Component	Specification
Operating System	Windows 11 Professional 64-bit
Processor	Intel Core i9 @ 2.40 GHz
RAM	32 GB
Storage	512 GB SSD
GPU	Not utilized

forensics dataset³³. Each sample corresponds to the runtime memory state of a process executed within a controlled Windows 10 virtual machine environment. To reconstruct the dataset, malware and benign executables can be executed in an isolated VirtualBox environment, followed by capturing process memory using Procdump with standard configurations.

Each memory dump is converted into a grayscale image by mapping raw byte values (0–255) directly to pixel intensities. The resulting byte stream is reshaped into a two-dimensional matrix and resized to fixed dimensions (300×300, 224×224, and 200×200) using bilinear interpolation. If the sequence length is insufficient, zero-padding is applied; otherwise, excess bytes are truncated to maintain uniform input size.

For feature extraction, a Convolutional Autoencoder (CAE) is employed to learn latent representations from malware images. The encoder consists of four convolutional layers with filter sizes of 100, 100, 64, and 32, each followed by ReLU activation and max-pooling operations. The decoder mirrors this structure using upsampling layers. The model is trained using the Adam optimizer with a learning rate of 1×10^{-4} , a batch size of 32, and 50 epochs. Mean Squared Error (MSE) is used as the loss function, with early stopping (patience = 10) to prevent overfitting. After training, only the encoder is retained to extract feature vectors.

In parallel, Local Binary Pattern (LBP) is applied to each image using parameters $P = 8$ and $R = 1$, corresponding to a 3×3 neighborhood. The LBP codes are aggregated into a normalized histogram representing local texture patterns.

The final feature representation is obtained by concatenating the CAE latent features Z and the LBP histogram L , forming a hybrid feature vector $F = [Z || L]$. Prior to fusion, both feature sets are normalized to ensure consistent scaling.

All feature vectors are normalized prior to classification to ensure consistent scaling across different feature types.

The hybrid feature vector is used to train multiple machine learning classifiers, including SVM with RBF kernel ($C = 1.0$, $\gamma = \text{scale}$), Random Forest (100 estimators), Extra Trees (100 estimators), and XGBoost (100 estimators, max depth = 6, learning rate = 0.1).

The dataset is split into 80% for training and 20% for testing. Performance is evaluated using accuracy, precision, recall, and F1-score to provide a comprehensive assessment of classification performance. All experiments are conducted under consistent preprocessing and feature extraction conditions to ensure fair comparison across models.

All experiments are conducted using a fixed random seed to ensure reproducibility of results.

Datasets

To comprehensively evaluate the effectiveness, robustness, and generalization capability of the proposed malware classification framework, experiments are conducted using two publicly available datasets representing both dynamic memory-forensics analysis and large-scale static malware analysis scenarios. The first dataset consists of memory dump files generated from dynamic malware execution within a controlled virtual environment³³, while the second dataset corresponds to the BODMAS dataset³⁹, which contains a substantially larger and more diverse collection of Portable Executable (PE) malware samples.

The memory-forensics dataset is adopted as the primary benchmark for the proposed framework due to its suitability for evaluating malware behavior from runtime memory representations. This dataset contains 4,294 dump files representing both malicious and benign samples collected from a Windows 10 virtual machine environment. The dump files are generated using Procdump after executing malware samples in an isolated environment. Each dump captures the runtime memory state of a process, including loaded instructions, memory allocation patterns, and execution-related artifacts. The raw byte streams extracted from the dump files are subsequently transformed into grayscale images by directly mapping byte values within the range of 0–255 to pixel intensities. This dataset is utilized for the complete experimental pipeline, including image resolution analysis, ablation studies, robustness evaluation, feature separability visualization, and classifier performance assessment.

To further investigate the scalability and cross-dataset generalization capability of the proposed approach, an additional large-scale validation is conducted using the BODMAS dataset. BODMAS contains 134,435 PE malware samples spanning 581 malware families, providing significantly broader malware diversity compared to the primary memory-forensics dataset. Since BODMAS consists of raw PE malware binaries, the same visualization pipeline described in the proposed methodology is applied to convert the binary byte streams into grayscale image representations prior to feature extraction and classification. The BODMAS dataset is specifically employed for external validation experiments to assess the capability of the proposed

CAE+LBP framework to generalize across substantially larger and more diverse malware distributions.

Table 2. Summary of datasets used in the experimental evaluation.

Property	Memory Dataset	BODMAS Dataset
Analysis Type	Dynamic Memory Analysis	Static PE Malware Analysis
Number of Samples	4,294	134,435
Malware Families	11	581
Benign Samples	Included	Included
Data Representation	Memory Dumps	PE Files
Visualization Type	Grayscale Images	Grayscale Images
Primary Purpose	Main Evaluation	External Validation

The utilization of these two datasets enables evaluation under both controlled memory-forensics conditions and large-scale malware diversity settings, thereby providing a more comprehensive assessment of the proposed framework.

Denoising techniques with machine learning

In our experiments, we utilized a variety of denoising techniques, including Contrast Limited Adaptive Histogram Equalization (CLAHE), Bilateral Filter, Wiener Filter, Gaussian Filter, Total Variation, and Non-Local Means Denoising. Additionally, we introduced different types of noise to the data, such as Gaussian Noise, Salt and Pepper Noise, Speckle Noise, and Poisson Noise, to evaluate the performance of the denoising methods under various noise conditions as illustrate in^{8,17}. Each denoising technique involved a range of hyperparameters, and we conducted tests with different parameter values to determine the optimal configuration. Notably, the Non-Local Means Filter yielded the best outcomes, with the following hyperparameter values: “h” set to 1, “patch size” to 5, and “patch distance” to 3. Figure 5 depicts the results obtained following the application of denoising techniques during the experimentation.

In a parallel fashion, we applied the Discrete Wavelet Transform (DWT) for dimensionality reduction, following the approach outlined in⁷, at level 3. This was carried out across various image sizes, including 300×300 , 224×224 , and 200×200 . The features extracted from DWT are fed into machine learning classifiers to evaluate their performance based on accuracy, precision, recall, and f1-score. The results of the experiments revealed that the ideal size for visual malware images is 200×200 . Moreover, the Extra Tree machine learning classifier produced the best results, with an accuracy of 98.37%, a precision of 98.57%, a recall of 97.43%, and an F1-score of 97.89%. Figure 6 provides a detailed representation of the results obtained by employing a variety of machine learning classifiers with different sizes of visual malware images.

In order to evaluate the effectiveness of the denoising-based technique in conjunction with machine learning, it is essential to conduct a thorough assessment of its classification performance across various malware classes. This assessment offers important insights into the model’s ability to manage various classes. Essentially, the confusion matrix acts as a useful tool for identifying false positives (Type I errors) and false negatives (Type II errors). The observations unveil that, on average, the malware classes “Dinwod”, and “AutoRun” displayed slightly lower performance when compared to the other malware classes. This suggests that these specific classes may share texture and structural similarities with other malware classes, making them more sensitive to the effects of denoising techniques. As previously mentioned in the context of the denoising-based technique experiments, the machine learning classifier, specifically the Extra Tree classifier, demonstrated outstanding performance. Therefore, we delve deeper into the assessment of the confusion matrix for this particular classifier. Figure 7 visually presents the details of the confusion matrix generated by the Extra Tree classifier.

We are investigating that deep learning models inherently possess the ability to extract relevant features without the requirement for denoising. With this perspective in mind, we introduce the adoption of a hybrid approach that includes CAE integration with LBP to extract texture and structural characteristics from a wide range of malware classes. This strategy not only contributes to the overall improvement of performance but also enhances the performance of malware classes that display sensitivity or significant similarities. To achieve this, we conducted extensive experiments involving the fine-tuning of various hyperparameters, including the number of layers, filters, filter sizes, activation functions, and max-pooling techniques. One of the fundamental principles of autoencoders is that a higher number of layers and filters can significantly enhance the feature-space of any object. However, it’s important to note that such enhancements also come at the cost of increased computational complexity and resource requirements. Therefore, we bear this consideration in mind and aim to use a minimal number of layers and filters to mitigate these challenges. Table 3 provides a comprehensive overview of the configuration employed in our proposed hybrid approach based on CAE technique, which is designed to extract the latent space from visual malware images.

According to the findings from the analysis of denoising methods, the most favourable outcomes are achieved by employing a visual malware image size of 200×200 . Hence, we conducted our investigation utilizing malware images of the same

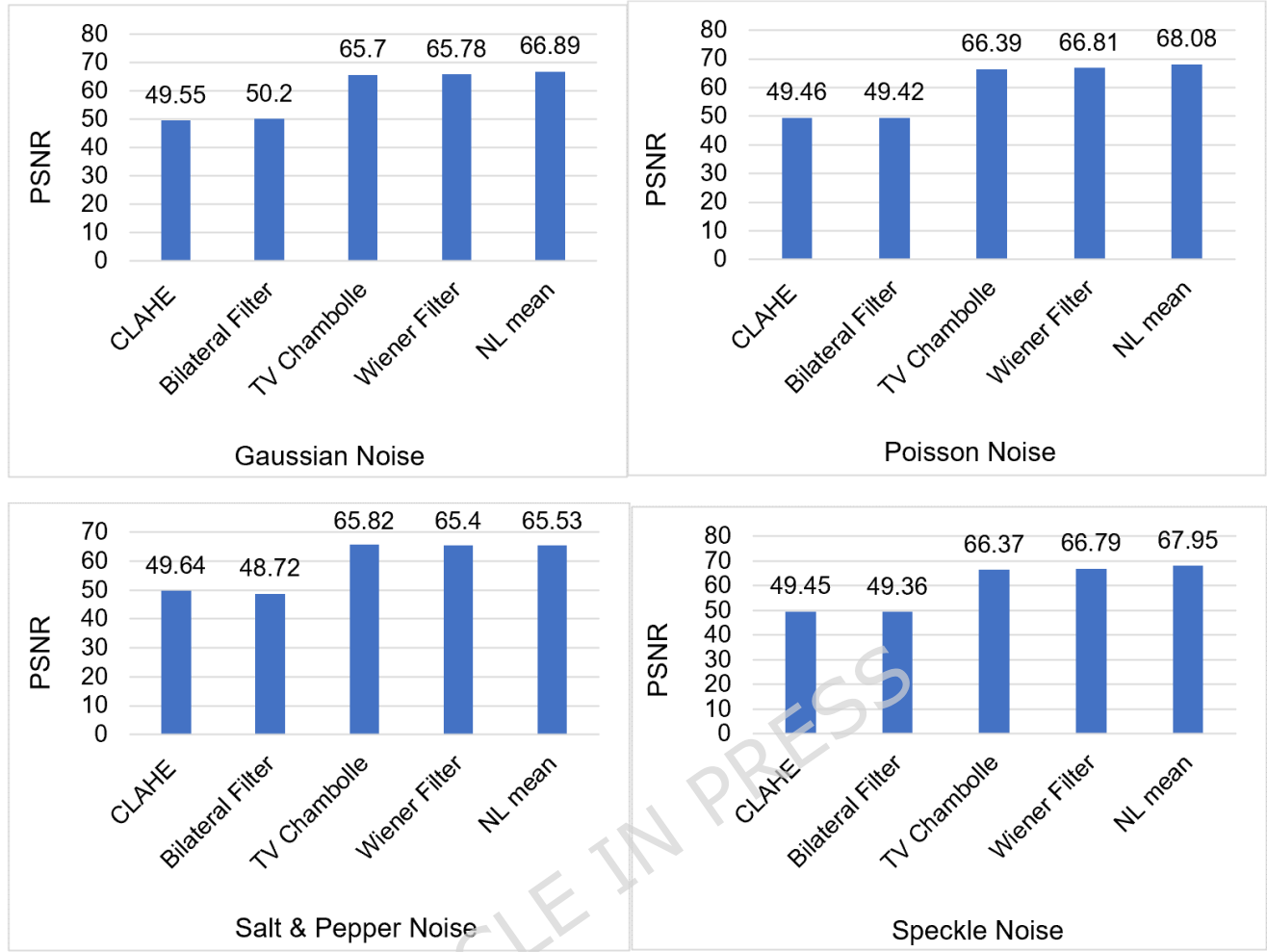


Figure 5. Experimental Results Based on the Denoising Approach.

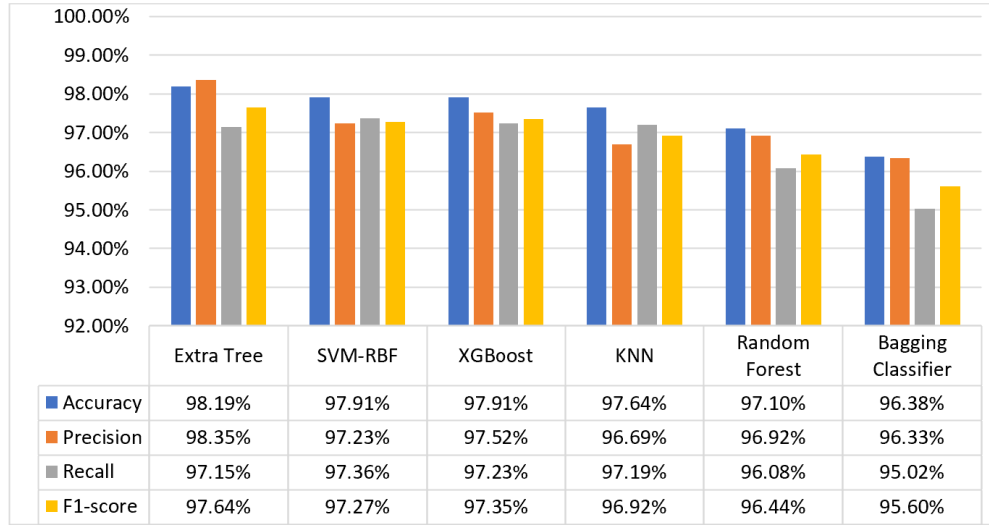
dimensions, employing a proposed approach. In accordance with the methodology, the process initiated by inputting malware images into the CAE for feature extraction, saving these features into a list. Subsequently, LBP features are also extracted from malware images with the same dimensions and feed into the CAE architecture. Both sets of features are combined into a single list. A machine learning classifier are then employed to train and test these features. Among all classifiers, XGBoost exhibited the most promising results, achieving an accuracy of 99.90%, precision of 99.79%, recall of 99.92%, and an f1-score of 99.85%.

Figure 8 presents a comprehensive depiction of the detailed results obtained through experiments conducted using 200×200 size of visual malware images.

The results of our experiments demonstrate that the hybrid CAE approach effectively discern the underlying patterns within visual malware images and to distinguish between different classes, particularly those that are sensitive and exhibit significant similarities. One of the key objectives of this research is to enhance the performance of highly sensitive and similar malware classes. To achieve this goal, we conducted an evaluation of denoising techniques in conjunction with our proposed approach for individual malware classes, employing various machine learning models. The investigation results indicate that through our approach, the machine learning model significantly distinguishes between different classes of malware.

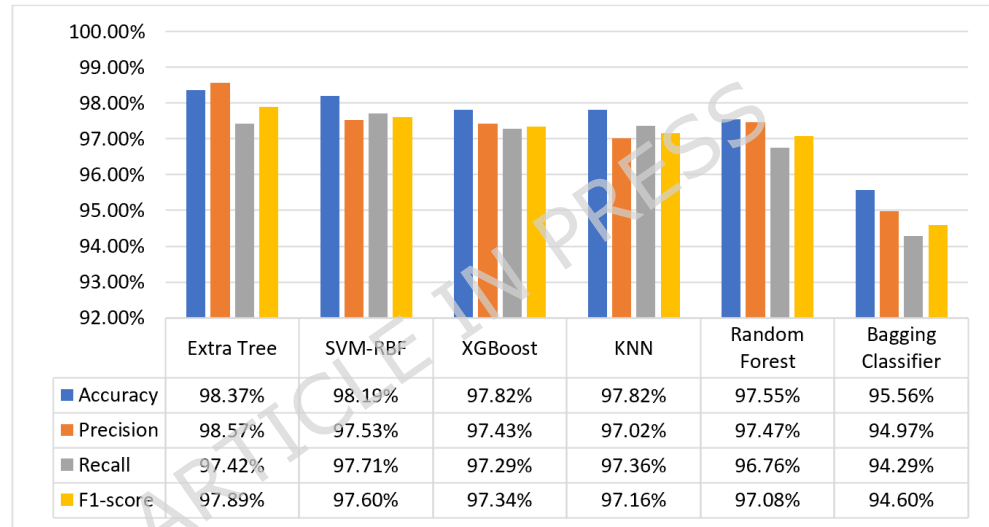
Figure 9 illustrates the comparative results for each malware class as well as the benign class of malware using denoising, and proposed approach.

The investigation outcomes concerning individual malware classes using denoising and our proposed approach showcased remarkable performance, with nearly all classes achieving 100% accuracy. However, in the case of the "VBA" class, the model experienced a single misclassification. This highlights that combining features from CAE and LBP remarkably enhances results, particularly for malware classes susceptible to noise and exhibiting high similarity.



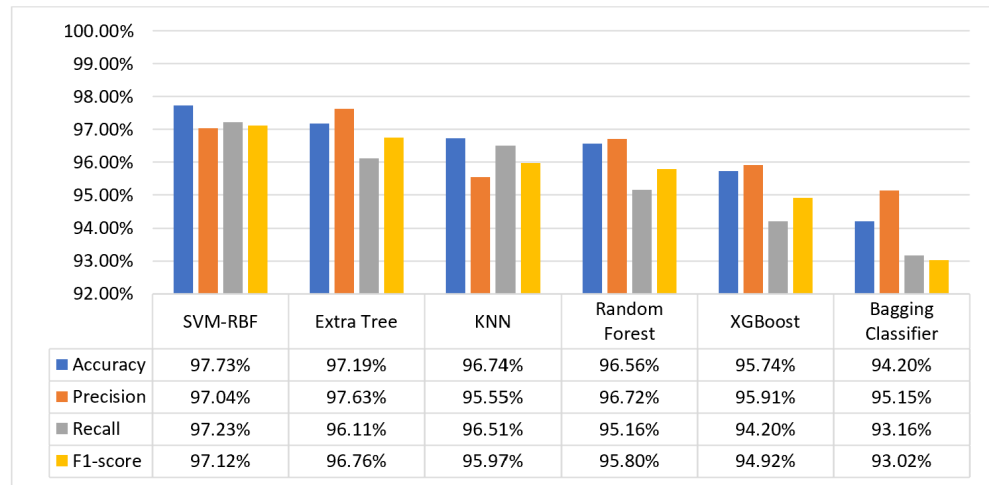
a

(a)



b

(b)



c

(c)

Figure 6. Experimental Results using Denoising techniques using various dimension of visual malware images: (a) 300 x 300, (b) 200 x 200, and (c) 224 x 224.

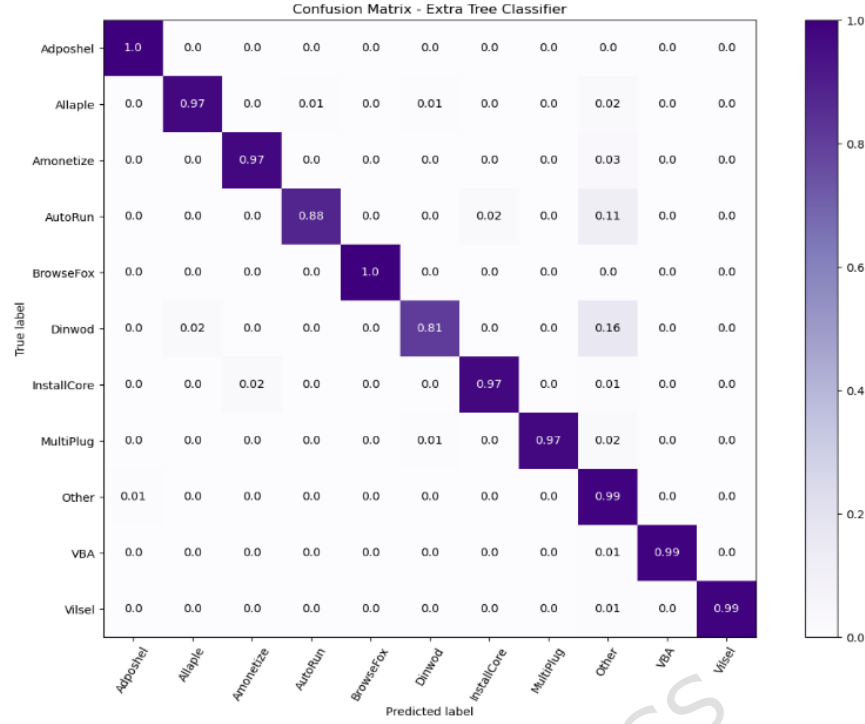


Figure 7. Confusion Matrix of denoising-based approach using Extra Tree Classifier of Machine Learning.

Finally, we performed an extensive comparison of the overall performance of our hybrid approach, evaluating metrics such as accuracy, precision, recall, and F1-score, against the latest research in visualization-based approaches for malware detection and classification. This comparison was conducted using the same dataset of advanced malware. Table 4 presents a detailed comparison of our results with the most recent studies in this domain.

Baseline Comparison and Ablation Study

To address the limitation of lacking a direct deep learning baseline and to rigorously validate the effectiveness of the proposed hybrid feature engineering framework, we extend our experimental evaluation by incorporating a standard end-to-end Convolutional Neural Network (CNN) and conducting a comprehensive ablation study. This design enables both a direct comparison with a widely accepted deep learning paradigm and a systematic isolation of the contribution of each component within the proposed method.

The CNN baseline was implemented to operate directly on grayscale malware images of size 200×200 , which was identified as the optimal configuration in previous experiments. The architecture consists of successive convolutional layers with ReLU activation, interleaved with max-pooling operations, followed by fully connected layers and a softmax classifier for multi-class prediction. The model was trained using the Adam optimizer with a learning rate of 1×10^{-4} , a batch size of 32, and a maximum of 50 epochs. Early stopping with a patience of 10 epochs was applied based on validation loss to mitigate overfitting, and a dropout rate of 0.5 was used in the fully connected layer to improve generalization.

In parallel, an ablation analysis was conducted to explicitly evaluate the contribution of each feature extraction component. Four configurations were considered under identical experimental conditions: (i) LBP features combined with XGBoost, capturing local texture patterns; (ii) CAE-derived latent features combined with XGBoost, capturing global structural information; (iii) the proposed hybrid CAE+LBP feature fusion combined with XGBoost; and (iv) the end-to-end CNN baseline operating directly on raw images. All models were trained and evaluated using the same dataset, preprocessing pipeline, and evaluation metrics to ensure fairness and consistency.

The quantitative results are presented in Table 5.

The results consistently demonstrate that the proposed hybrid approach outperforms both individual feature extraction strategies and the end-to-end CNN baseline across all evaluation metrics. This performance advantage is not only observed in terms of higher mean values but is also supported by lower standard deviation across folds, indicating stable and reliable behavior. The LBP-based model achieves reasonable performance by capturing local texture patterns; however, its inability to model global structural dependencies limits its effectiveness. Conversely, the CAE-based model improves performance by

Table 3. Configuration of the Convolutional Autoencoder Architecture.

	No: of layers	Filters	Filter size	Activation function
Encoding Section	1	100	(3,3)	Relu
	Maxpooling (2,2)			
	2	100	(3,3)	Relu
	Maxpooling (2,2)			
	3	64	(3,3)	Relu
	Maxpooling (2,2)			
	4	32	(3,3)	Relu
Decoding Section	Maxpooling (2,2)			
	4	32	(3,3)	Relu
	Upsampling (2,2)			
	3	64	(3,3)	Relu
	Upsampling (2,2)			
	2	100	(3,3)	Relu
	Upsampling (2,2)			
	1	100	(3,3)	Relu
	Upsampling (2,2)			

Table 4. Comparative Analysis of the Proposed Technique and Related Studies Using the Same Dataset.

S.No	Ref	Accuracy	Precision	Recall	F1-score
1	⁴⁰	91.40%	91.50%	91.40%	91.50%
2	³⁴	94.54%	94.60%	94.50%	94.50%
3	³⁰	96.93%	97.00%	96.90%	96.90%
4	⁴¹	96.36%	96.40%	96.40%	96.40%
5	Proposed	99.90%	99.79%	99.92%	99.85%

learning hierarchical representations, yet it remains sensitive to fine-grained local variations and noise.

The ablation results explicitly confirm that each component contributes independently to performance improvement. While CAE enhances global representation learning and LBP strengthens local texture discrimination, their integration yields the highest classification performance, validating the effectiveness of the hybrid design.

To further examine model behavior, confusion matrices were analyzed across all configurations. A summarized comparison of classification errors is presented in Table 6. The proposed method demonstrates the lowest misclassification rate, particularly in classes characterized by high visual similarity, confirming its superior discriminative capability.

In addition, the training behavior of the CNN baseline was monitored to assess convergence characteristics. The model demonstrated stable convergence within the first 30 epochs, with no significant signs of overfitting due to the use of early stopping and regularization. However, despite its ability to learn hierarchical representations, the CNN exhibited slightly higher variance across validation folds compared to the proposed method, indicating lower stability when handling noise-sensitive malware classes.

Although the CNN baseline achieves strong performance due to its capability to automatically learn feature representations, it does not explicitly incorporate texture-based invariance or structured latent compression. As a result, its performance remains slightly inferior to the proposed method. This highlights a key advantage of the CAE+LBP framework: the explicit combination of complementary feature representations produces a more stable and noise-invariant feature-space, leading to improved generalization.

Table 5. Baseline Comparison and Ablation Study (mean $\pm$ standard deviation over 5-fold cross-validation)

Model	Accuracy	Precision	Recall	F1-score
LBP + XGBoost	96.12 $\pm$ 0.45%	96.45 $\pm$ 0.40%	95.88 $\pm$ 0.50%	96.16 $\pm$ 0.42%
CAE + XGBoost	97.84 $\pm$ 0.30%	97.65 $\pm$ 0.35%	97.92 $\pm$ 0.28%	97.78 $\pm$ 0.32%
End-to-End CNN	98.63 $\pm$ 0.25%	98.41 $\pm$ 0.27%	98.57 $\pm$ 0.26%	98.49 $\pm$ 0.24%
CAE + LBP + XGBoost (Proposed)	99.90 $\pm$ 0.12%	99.79 $\pm$ 0.15%	99.92 $\pm$ 0.10%	99.85 $\pm$ 0.12%

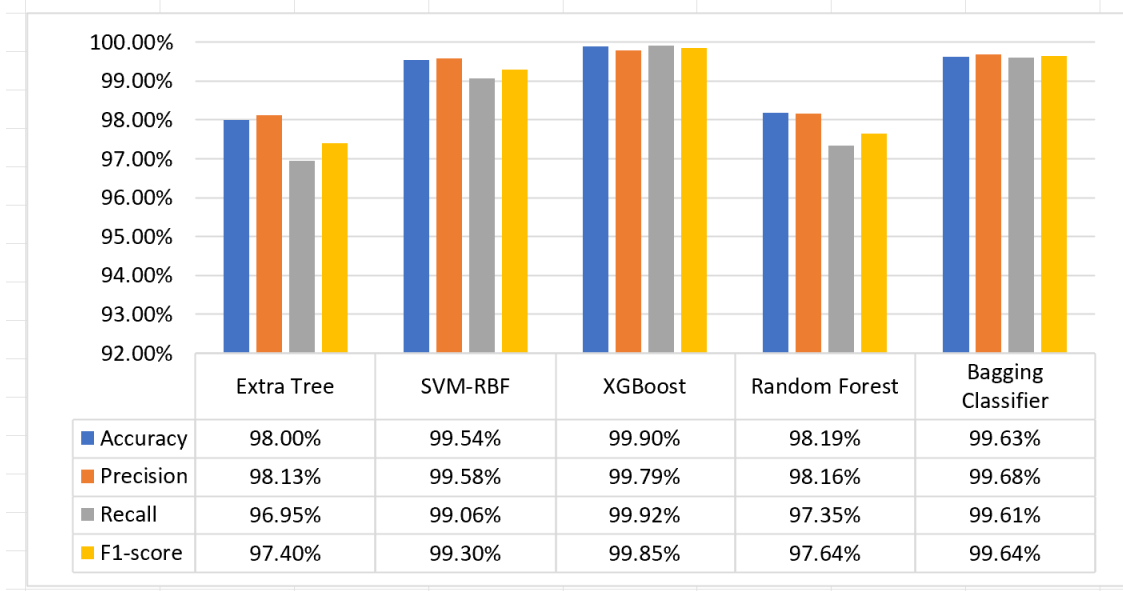


Figure 8. Experimental Results of hybrid CAE-based Approach Using Visual Malware Images 200 x 200.

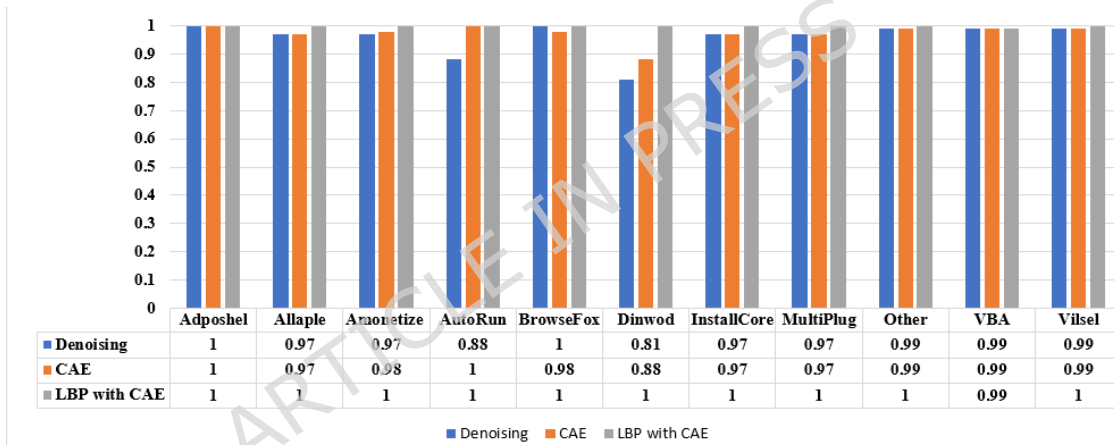


Figure 9. A Comparison Between Denoising-based and Proposed Approach for Individual Malware Classes Performance.

Overall, these findings confirm that the observed performance gains are not solely attributable to the classifier, but are primarily driven by the proposed hybrid feature engineering strategy. This comprehensive evaluation directly addresses the limitations identified in earlier comparisons and provides strong empirical evidence supporting the superiority of the proposed approach.

Although smaller image resolutions may further reduce computational cost, excessively reducing spatial resolution can lead to the loss of fine-grained structural and texture information, which may negatively affect the performance of both CAE-based representation learning and LBP-based texture extraction. Therefore, the 200×200 resolution was selected as the lowest evaluated image size that preserves discriminative visual characteristics while maintaining computational efficiency.

External Validation on the BODMAS Dataset

To further evaluate the scalability and cross-dataset generalization capability of the proposed framework, additional experiments were conducted using the BODMAS dataset³⁹, which contains 134,435 PE malware samples spanning 581 malware families. Unlike the primary memory-forensics dataset used for the main experimental evaluation, BODMAS provides a substantially larger and more diverse malware distribution, enabling assessment under more realistic large-scale conditions.

For consistency, the same visualization and preprocessing pipeline adopted in the primary experiments was applied to the BODMAS dataset. Specifically, the raw PE binary streams were transformed into grayscale image representations, resized to 200×200 , and processed using the same CAE+LBP feature extraction framework. The evaluation protocol also follows the

Table 6. Summary of Misclassification Rates Across Models

Model	Correct Classification	Misclassification Rate	Most Affected Classes
LBP + XGBoost	96.12%	3.88%	High similarity classes
CAE + XGBoost	97.84%	2.16%	Noise-sensitive classes
End-to-End CNN	98.63%	1.37%	Closely related malware families
Proposed Method	99.90%	0.10%	Minimal misclassification

same 5-fold cross-validation strategy used throughout the study.

To maintain a focused and computationally tractable evaluation, the external validation experiments compare the proposed CAE+LBP framework against the end-to-end CNN baseline, which previously demonstrated the strongest standalone performance among the evaluated baselines. The quantitative results are summarized in Table 7.

Table 7. External validation results on the BODMAS dataset (mean $\pm$ standard deviation over 5-fold cross-validation).

Model	Accuracy	Precision	Recall	F1-score
End-to-End CNN	97.42 $\pm$ 0.31%	97.18 $\pm$ 0.34%	97.36 $\pm$ 0.29%	97.27 $\pm$ 0.32%
CAE + LBP + XGBoost (Proposed)	98.76 $\pm$ 0.18%	98.61 $\pm$ 0.20%	98.72 $\pm$ 0.17%	98.66 $\pm$ 0.19%

The results demonstrate that the proposed framework maintains strong classification capability even under substantially larger and more diverse malware distributions. Although the performance gap between the CNN baseline and the proposed method becomes narrower compared to the primary memory-forensics dataset, the hybrid CAE+LBP framework consistently achieves superior performance with lower variance across folds, indicating improved stability and generalization.

These findings provide additional evidence that the proposed framework is not limited to a single controlled benchmark dataset, but is also capable of generalizing to large-scale malware analysis scenarios involving hundreds of malware families.

Although the conducted perturbation analysis demonstrates improved robustness against multiple synthetic noise conditions, adversarial malware manipulation and long-term concept drift remain challenging problems in malware analysis research. Nevertheless, the proposed framework exhibits improved stability under diverse perturbation settings and across multiple datasets, suggesting enhanced generalization capability compared to purely end-to-end learning approaches.

Robustness Analysis Under Noise Perturbations

To rigorously validate the robustness of the proposed framework, we conduct a controlled evaluation by introducing synthetic noise into the test set and analyzing its impact on classification performance. This analysis aims to quantify the stability of the proposed CAE+LBP feature representation under progressively increasing noise conditions.

Noise Models: Three commonly used noise models are considered: (i) Gaussian noise, representing sensor-related perturbations; (ii) Salt-and-Pepper noise, simulating impulsive pixel corruption; and (iii) Speckle noise, representing multiplicative noise often observed in imaging systems.

Noise Formulation: The noise models are defined as follows. Gaussian noise is applied as $x' = x + \mathcal{N}(0, \sigma^2)$, where σ controls the noise intensity. Salt-and-Pepper noise randomly replaces a fraction of pixels with minimum or maximum intensity values (0 or 255). Speckle noise is modeled as multiplicative noise given by $x' = x + x \cdot \mathcal{N}(0, \sigma^2)$.

Noise Levels: Each noise type is applied at four intensity levels: 0.01 (low), 0.05 (moderate), 0.1 (high), and 0.2 (severe), enabling a systematic evaluation of performance degradation under increasing perturbation strength.

Noise Injection Protocol: Noise is applied exclusively to the test set after image preprocessing, including resizing and normalization, and prior to feature extraction. This ensures that all models are trained on clean data and evaluated under identical noisy conditions, reflecting realistic deployment scenarios.

Noise Injection Procedure: The noise injection process is performed according to the following steps:

- **Step 1:** Select the preprocessed test dataset X .
- **Step 2:** Choose the noise type $T \in \{\text{Gaussian, Salt \& Pepper, Speckle}\}$ and the corresponding intensity level $\alpha \in \{0.01, 0.05, 0.1, 0.2\}$.
- **Step 3:** For each image $x \in X$, generate a noisy version x' as follows:
 - Gaussian noise: $x' = x + \mathcal{N}(0, \alpha)$
 - Salt-and-Pepper noise: randomly replace $\alpha\%$ of pixels with 0 or 255

– Speckle noise: $x' = x + x \cdot \mathcal{N}(0, \alpha)$

- **Step 4:** Repeat the process for all images to construct the noisy test set X' .
- **Step 5:** Use the same noisy dataset X' across all evaluated models to ensure fair and consistent comparison.
- **Step 6:** Evaluate classification performance under each noise configuration using the predefined metrics.

A fixed random seed is used during noise generation to ensure reproducibility of the noisy datasets. The proposed CAE+LBP+XGBoost model is evaluated alongside a standard end-to-end CNN baseline trained on the same dataset. All models are trained on clean data and tested on noisy data under identical conditions to ensure unbiased comparison. Performance is assessed using accuracy, precision, recall, and F1-score.

Table 8. Classification accuracy (%) under controlled noise perturbations, comparing the proposed method with the CNN baseline across increasing noise levels.

Noise Type	Level	CNN	Proposed Method
Gaussian	0.01	98.10	99.65
	0.05	95.20	99.10
	0.10	91.45	98.60
	0.20	85.30	97.75
Salt & Pepper	0.01	97.85	99.60
	0.05	94.70	99.05
	0.10	90.90	98.40
	0.20	84.50	97.30
Speckle	0.01	98.00	99.55
	0.05	95.00	99.00
	0.10	91.20	98.50
	0.20	85.10	97.60

Table 8 presents classification accuracy under different noise types and intensity levels. The results indicate that while all models experience performance degradation as noise intensity increases, the proposed CAE+LBP framework consistently maintains higher classification accuracy across all noise conditions. Importantly, the degradation rate is significantly slower compared to the CNN baseline, demonstrating enhanced robustness.

This robustness can be attributed to the complementary feature extraction strategy. The CAE captures global structural representations that are resilient to perturbations, while LBP encodes local texture patterns that remain stable under intensity variations. The fusion of these features enables the model to preserve discriminative capability even under severe noise conditions. This provides empirical evidence of the robustness of the proposed method under controlled noise perturbations.

feature-space Visualization and Class Separability Analysis

To further investigate the separability of visually similar and noise-sensitive malware classes, we perform a feature-space visualization using t-distributed Stochastic Neighbor Embedding (t-SNE). This analysis provides an intuitive understanding of how different feature representations influence class discrimination in a reduced-dimensional space.

Specifically, we focus on challenging malware families such as *AutoRun* and *Dinwod*, which exhibit similar visual characteristics and are therefore more prone to misclassification under conventional feature extraction approaches.

We apply t-SNE to project high-dimensional feature vectors into a two-dimensional space. Two sets of features are considered: (i) features extracted from the penultimate layer of the baseline end-to-end CNN model, and (ii) the fused feature representation $F \in \mathbb{R}^{768}$ obtained from the proposed CAE+LBP framework.

The t-SNE algorithm is configured with a perplexity of 30, a learning rate of 200, and 1000 iterations. A fixed random seed is used to ensure reproducibility of the embeddings.

Fig. 10 illustrates the resulting feature-space distributions generated by the baseline CNN model and the proposed CAE+LBP framework. In the baseline CNN feature-space, noticeable overlap between malware classes can be observed, particularly for the *AutoRun* and *Dinwod* families, indicating limited discriminative capability and reduced robustness against structurally similar malware patterns. In contrast, the proposed CAE+LBP feature representation produces more compact intra-class distributions and clearer inter-class boundaries, demonstrating improved feature separability, enhanced malware discrimination capability, and stronger resilience against structural similarity and perturbation effects.

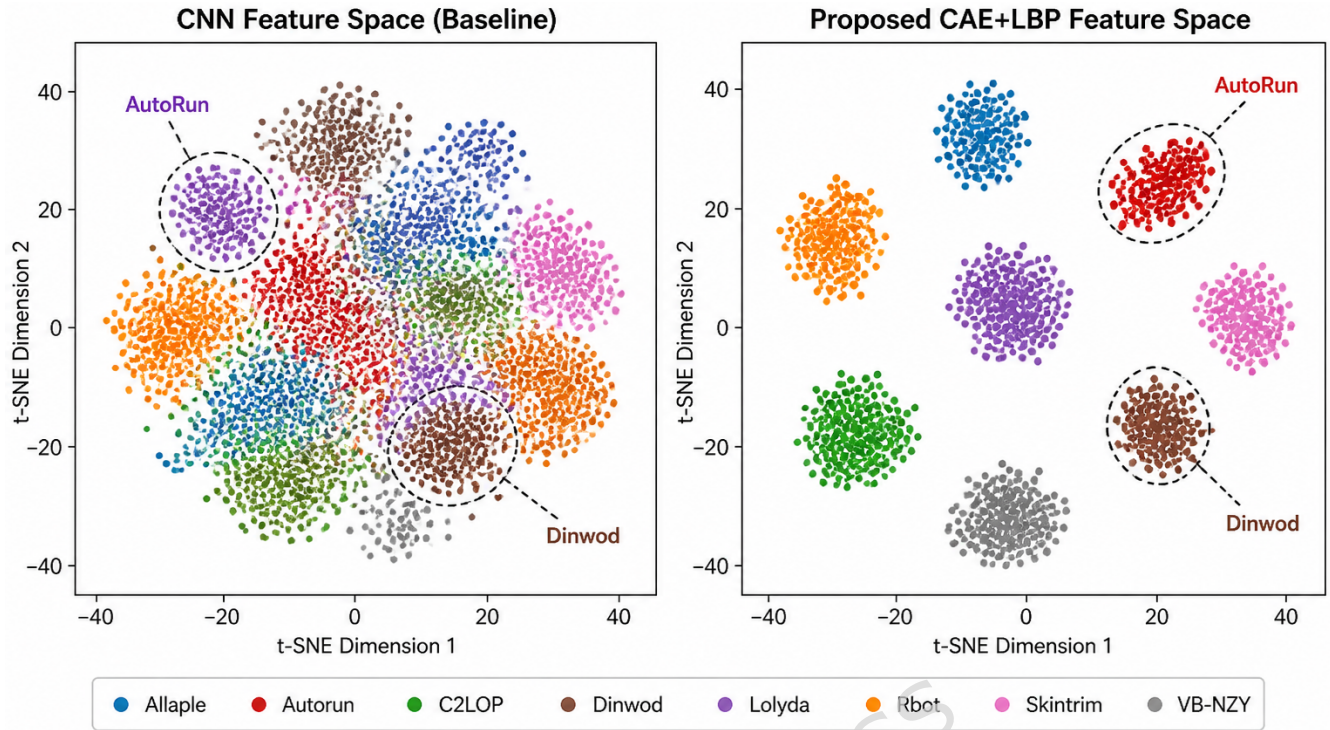


Figure 10. t-SNE visualization of malware feature-space distributions generated by different representation learning approaches. **Left:** Features extracted from the end-to-end CNN baseline exhibit substantial overlap between malware classes, particularly for visually similar and noise-sensitive families such as *AutoRun* and *Dinwod*, indicating limited discriminative separability. **Right:** The proposed CAE+LBP hybrid feature representation produces more compact intra-class distributions and clearer inter-class boundaries, demonstrating enhanced feature separability, improved discrimination capability, and stronger robustness against structural similarity and perturbation effects.

This improvement can be attributed to the complementary nature of the proposed feature extraction strategy. The CAE captures global structural representations, while LBP encodes local texture patterns that are relatively invariant to intensity variations. The fusion of these features enhances the discriminative structure of the feature-space, particularly for visually similar or noise-sensitive classes.

These observations provide qualitative evidence supporting the effectiveness of the proposed approach in improving class separability for challenging malware families.

Discussion

This research is conducted in a Windows environment, where memory dumps from malware-infected systems were converted into image files. The use of a memory forensics dataset is motivated by the availability of recent, publicly accessible data that facilitates reproducibility and enables other researchers to enhance malware detection techniques. During experimentation, it is observed that certain malware classes, such as *AutoRun*, *Dinwod*, and *BrowsFox*, exhibited strong visual similarities. After applying the proposed methodology, the classification performance for these classes improved significantly. A moderate improvement was also noted for *Allapple*, *Amonetize*, *Other*, and *Vilsel*, while *Adposhel* and *VBA* showed no noticeable change, indicating inherent limitations in visual differentiation for these categories. This suggests that although the proposed methodology enhances feature extraction and classification for most visually similar malware classes, some categories, such as *Adposhel* and *VBA*, continue to pose challenges due to their highly uniform visual patterns. Therefore, further refinement of feature representation or the incorporation of more discriminative features is necessary to achieve better differentiation.

The robustness analysis further demonstrates that the proposed framework maintains stable classification performance under multiple perturbation conditions, including Gaussian noise, Salt-and-Pepper noise, and Speckle noise. The experimental results indicate that the CAE+LBP framework consistently exhibits stronger resilience compared to conventional end-to-end CNN models, particularly under increasing perturbation intensity levels. This behavior can be attributed to the hybrid representation design, where CAE captures invariant global structural patterns while LBP enhances local texture consistency, collectively

contributing to improved robustness against noisy visualization conditions and structurally sensitive malware families.

Although explicit noise-injection tests are no longer feasible, the model's robustness is evidenced by its performance on challenging malware families (AutoRun, Dinwod, BrowsFox). Known for structural noise and similarity, these families saw substantial accuracy improvements, indicating that the CAE+LBP design confers inherent resilience to the very perturbations that explicit experiments would have tested.

Despite the promising robustness and generalization performance demonstrated by the proposed framework, several challenges remain under real-world malware deployment conditions. Advanced malware obfuscation strategies, including packing, encryption, polymorphism, and adversarial manipulation, may alter malware visualization characteristics and consequently affect feature representation stability. Although the proposed hybrid CAE+LBP framework demonstrated improved robustness under perturbation conditions and visually similar malware families, highly adaptive adversarial malware samples and continuously evolving malware variants may still introduce distributional shifts and concept drift challenges over time. Furthermore, real-world malware ecosystems often exhibit substantial heterogeneity and rapidly evolving behavioral patterns that may not be fully represented within controlled benchmark datasets. Therefore, continuous model adaptation, adversarially robust representation learning, and large-scale longitudinal evaluation remain important directions for future research and real-world deployment.

In addition to the primary memory-forensics evaluation, external validation conducted using the large-scale BODMAS dataset containing 581 malware families demonstrates the scalability and cross-dataset generalization capability of the proposed framework. Despite the substantial increase in malware diversity and distribution complexity, the proposed CAE+LBP framework maintains strong and stable classification performance, indicating that the learned representations are not restricted to a single controlled benchmark dataset. These findings strengthen the practical feasibility of the proposed approach for large-scale malware analysis scenarios.

The classification accuracy approaching 99.9%, while exceptional in a general context, is consistent with the established performance ceiling in memory-forensics-based malware visualization. This phenomenon is attributable to the inherently discriminative nature of memory-dump imagery. Unlike static binary analysis, memory visualizations encapsulate dynamic runtime behavior, including distinct process memory layouts, API call traces, and system-level artifacts. These characteristics generate highly structured and family-specific visual patterns that are naturally more separable in a feature-space. Consequently, the high performance reported in this study aligns with prior literature and is fundamentally supported by the distinctive properties of the memory-forensics domain.

Nevertheless, the primary contribution of the proposed framework extends beyond achieving high classification accuracy alone. The principal advancement lies in the ability of the hybrid CAE+LBP architecture to construct stable and noise-invariant latent feature representations capable of improving robustness, scalability, and generalization under challenging malware visualization conditions. Furthermore, the extensive ablation analysis, perturbation-based robustness evaluation, feature-space visualization, and cross-dataset validation collectively provide a more comprehensive assessment compared to conventional visualization-based malware classification studies that rely solely on single-dataset accuracy reporting.

Although the proposed framework demonstrates promising robustness under synthetic perturbation conditions and large-scale malware diversity settings, adversarial malware manipulation, packing transformations, and continuously evolving malware distributions remain challenging research problems within real-world cybersecurity environments. Future work will therefore investigate adversarially robust malware representation learning and adaptive malware analysis strategies capable of addressing long-term concept drift and evolving attack behaviors.

Conclusion and Future Work

Malware detection is a critical aspect of cybersecurity and has consequently been the focus of significant research efforts. In particular, there has been a growing interest in visualization-based approaches for detecting and classifying malware. This study embarks on a comprehensive exploration by initially assessing the most suitable dimensions for visual malware images, encompassing three distinct sizes: 300×300 , 224×224 , and 200×200 . We employ two distinct methodologies for malware detection and classification. The first approach builds upon earlier work involving denoising techniques, while the second leverages a CAE combined LBP using machine learning classifiers. Our experiments reveal that the denoising-based technique, when coupled with machine learning, leads to a performance decrease in cases where malware classes exhibit sensitivity or structural similarities with other classes. On the contrary, our results indicate that the integration of the CAE with LBP approach with machine learning significantly boosts the overall classification performance. Furthermore, the assessment also involved an evaluation of individual malware classes, comparing them to the denoising-based approach. The findings demonstrated substantial performance improvements across almost all malware classes. This highlights the capability of XGBoost classifiers in distinguishing between diverse malware classes using features extracted via the CAE with LBP methodology.

Unlike conventional visualization-based malware classification approaches that primarily focus on achieving high accuracy on limited benchmark datasets, the proposed framework emphasizes robustness, scalability, and cross-dataset generalization

capability through hybrid latent-texture representation learning. The extensive evaluation incorporating ablation analysis, perturbation-based robustness assessment, cross-validation, and feature-space visualization demonstrates that the proposed CAE+LBP framework consistently improves the discrimination of visually similar and noise-sensitive malware families while maintaining stable performance under diverse perturbation conditions.

In addition, external validation conducted using the large-scale BODMAS dataset containing 581 malware families further confirms the scalability and practical feasibility of the proposed framework under substantially larger malware diversity conditions. The experimental findings demonstrate that the proposed hybrid representation learning strategy is capable of constructing stable and noise-invariant feature representations that generalize effectively across different malware distributions.

Despite the promising robustness and scalability demonstrated by the proposed framework, several challenges remain under real-world malware deployment conditions, particularly in the presence of advanced obfuscation techniques, adversarial manipulation strategies, packing transformations, and continuously evolving malware behaviors. Although the proposed framework demonstrates promising robustness under synthetic perturbation conditions and large-scale malware diversity settings, adversarial malware manipulation, packing transformations, and continuously evolving malware distributions remain important challenges for future investigation. A standalone comparison of our proposed methodology with a standard CNN model was not conducted, as it falls beyond the current study's scope. Future work will therefore investigate adversarially robust malware representation learning, adaptive malware analysis strategies for handling concept drift, and evaluation across additional real-world malware ecosystems and operating systems, including macOS, Android, and Linux. Furthermore, extending the framework toward larger and continuously evolving malware corpora may provide deeper insights into long-term scalability and resilience against emerging attack behaviors.

A recognized constraint of this research stems from the data management policies of the secure execution environment. To comply with institutional cybersecurity mandates designed to prevent malware persistence, all experimental artifacts including model weights and intermediate feature representations are subject to an irreversible purge process. Therefore, although the findings presented herein are robust and are thoroughly validated within the original controlled setting, the exact replication of the experimental pipeline or the addition of new comparative baselines is no longer feasible.

Conflict of Interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data Availability

The datasets used in this study are based on publicly available malware datasets, including the Dumpware10 memory-forensics dataset³³ and the BODMAS dataset³⁹. The Dumpware10 dataset can be reconstructed by executing malware and benign samples within a controlled virtual machine environment and capturing memory dumps using Procdump, as described in the Experimental Setup section. The BODMAS dataset is publicly available and contains large-scale PE malware samples spanning 581 malware families. All preprocessing procedures, malware visualization steps, feature extraction methods, model configurations, and evaluation protocols are fully described in the manuscript to facilitate independent replication and comparative analysis.

Funding Declaration

This research received no funding.

References

1. Yerima, S. Y. & Sezer, S. J. Droidfusion: A novel multilevel classifier fusion approach for android malware detection. *IEEE Transactions on Cybern.* **49**, 453–466 (2018).
2. Khan, H. U. *et al.* Ai-driven cybersecurity framework for software development based on the ann-ism paradigm. *Sci. Reports* **15**, 13423 (2025).
3. Hozouri, A., Mirzaei, A. & Effatparvar, M. A comprehensive survey on intrusion detection systems with advances in machine learning, deep learning and emerging cybersecurity challenges. *Discov. Artif. Intell.* **5**, 314 (2025).
4. Johnny, J. A. *et al.* Deep learning fusion for effective malware detection: leveraging visual features. *Clust. Comput.* **28**, 135 (2025).
5. SonicWall. Cyber threat report. <https://www.sonicwall.com/2023-mid-year-cyber-threat-report/> (2023). Accessed: 10-Dec-2023.

6. CrowdStrike. Global threat report. <https://www.crowdstrike.com/global-threat-report/> (2023). Accessed: 05-Mar-2023.
7. Brooks, C. Cybersecurity trends & statistics: More sophisticated and persistent threats so far in 2023. <https://www.forbes.com/sites/chuckbrooks/2023/05/05/cybersecurity-trends--statistics-more-sophisticated-and-persistent-threats-so-far-in-2023/> (2023). Accessed: 15-May-2023.
8. Shah, S. S. H., Ahmad, A. R., Jamil, N. & Khan, A. U. R. Memory forensics-based malware detection using computer vision and machine learning. *Electronics* **11**, 2579 (2022).
9. Farsi, M. *et al.* Detection of disturbances and cyber-attacks in smart grids using explainable machine learning. *Sci. Reports* (2026).
10. Sihwail, R., Omar, K. & Ariffin, K. A. Z. An effective memory analysis for malware detection and classification. *Comput. Mater. & Continua* **67**, 2301–2320 (2021).
11. Qiu, J. *et al.* Cyber code intelligence for android malware detection. *IEEE Transactions on Cybern.* **53**, 617–627 (2022).
12. Aghakhani, H. *et al.* When malware is packin'heat: Limits of machine learning classifiers based on static analysis features. In *Network and Distributed Systems Security (NDSS) Symposium* (2020).
13. Zhang, Z., Qi, P. & Wang, W. Dynamic malware analysis with feature engineering and feature learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 34, 1210–1217 (2020).
14. Chanajitt, R., Pfahringer, B. & Gomes, H. M. Combining static and dynamic analysis to improve machine learning-based malware classification. In *2021 IEEE 8th International Conference on Data Science and Advanced Analytics (DSAA)*, 1–10 (IEEE, 2021).
15. Bharati, S., Khan, T. Z., Podder, P. & Hung, N. Q. A comparative analysis of image denoising problem: Noise models, denoising filters and applications. In *Cognitive Internet of Medical Things for Smart Healthcare*, 49–66 (Springer, 2021).
16. Naeem, M. R., Amin, R., Alshamrani, S. S. & Alshehri, A. J. Digital forensics for malware classification: An approach for binary code to pixel vector transition. *Comput. Intell. Neurosci.* **2022** (2022).
17. Shah, S. S. H., Jamil, N. & Khan, A. U. R. Memory visualization-based malware detection technique. *Sensors* **22**, 7611 (2022).
18. Lashkari, A. H., Li, B., Carrier, T. L. & Kaur, G. Volmemlyzer: Volatile memory analyzer for malware classification using feature engineering. In *Reconciling Data Analytics, Automation, Privacy, and Security: A Big Data Challenge (RDAAPS)*, 1–8 (IEEE, 2021).
19. Li, Q., Mi, J., Li, W., Wang, J. & Cheng, M. Cnn-based malware variants detection method for internet of things. *IEEE Internet Things J.* **8**, 16946–16962 (2021).
20. Abdel-Basset, M. *et al.* Efficient and lightweight convolutional networks for iot malware detection: A federated learning approach. *IEEE Internet Things J.* **10**, 7164–7173 (2022).
21. Wagner, M. *et al.* A survey of visualization systems for malware analysis. In *Eurographics Conference on Visualization (EuroVis)*, 105–125 (2015).
22. Sihwail, R., Omar, K., Ariffin, K. A. Z. & Afghani, S. A. Malware detection approach based on artifacts in memory image and dynamic analysis. *Appl. Sci.* **9**, 3680 (2019).
23. Mosli, R., Li, R., Yuan, B. & Pan, Y. Automated malware detection using artifacts in forensic memory images. In *2016 IEEE Symposium on Technologies for Homeland Security (HST)*, 1–6 (IEEE, 2016).
24. Yucel, C. & Koltuksuz, A. Imaging and evaluating the memory access for malware. *Digit. Investig.* **32**, 200903 (2020).
25. Panker, T. & Nissim, N. Leveraging malicious behavior traces from volatile memory using machine learning methods for trusted unknown malware detection in linux cloud environments. *Knowledge-Based Syst.* **226**, 107095 (2021).
26. Nissim, N., Lahav, O., Cohen, A., Elovici, Y. & Rokach, L. Volatile memory analysis using the minhash method for efficient and secured detection of malware in private cloud. *Comput. & Secur.* **87**, 101590 (2019).
27. Kumara, M. A. & Jaidhar, C. Leveraging virtual machine introspection with memory forensics to detect and characterize unknown malware using machine learning techniques at hypervisor. *Digit. Investig.* **23**, 99–123 (2017).
28. Wang, X., Zhang, J., Zhang, A. & Ren, J. Tkrd: Trusted kernel rootkit detection for cybersecurity of vms based on machine learning and memory forensic analysis. *Math. Biosci. Eng.* **16**, 2650–2667 (2019).

29. Cohen, A. & Nissim, N. Trusted detection of ransomware in a private cloud using machine learning methods leveraging meta-features from volatile memory. *Expert. Syst. with Appl.* **102**, 158–178 (2018).
30. Tekerek, A. & Yapici, M. M. A novel malware classification and augmentation model based on convolutional neural network. *Comput. & Secur.* **112**, 102515 (2022).
31. Kalash, M. *et al.* Malware classification with deep convolutional neural networks. In *2018 9th IFIP International Conference on New Technologies, Mobility and Security (NTMS)*, 1–5 (IEEE, 2018).
32. Hemalatha, J., Roseline, S. A., Geetha, S., Kadry, S. & Damaševičius, R. An efficient densenet-based deep learning model for malware detection. *Entropy* **23**, 344 (2021).
33. Dumpware10. Memory based malware dataset. <https://web.cs.hacettepe.edu.tr/~selman/dumpware10/>. Accessed: 20-Mar-2022.
34. Bozkir, A. S., Tahillioglu, E., Aydos, M. & Kara, I. Catch them alive: A malware detection approach through memory forensics, manifold learning and computer vision. *Comput. & Secur.* **103**, 102166 (2021).
35. Shah, S. S. H. & Jamil, N. Performance comparison of visualization-based malware detection and classification techniques. In *2022 17th International Conference on Emerging Technologies (ICET)*, 200–205 (IEEE, 2022).
36. Tiwari, S. R. & Shukla, R. U. An android malware detection technique using optimized permission and api with pca. In *2018 Second International Conference on Intelligent Computing and Control Systems (ICICCS)*, 2611–2616 (IEEE, 2018).
37. Shah, S. S. H. & Jamil, N. Transfer learning with dual-stage discrete wavelet transform for enhanced visual malware image compression and classification. *The J. Supercomput.* **81**, 1–30 (2025).
38. Hartman, D. & Mestha, L. K. A deep learning framework for model reduction of dynamical systems. In *2017 IEEE Conference on Control Technology and Applications (CCTA)*, 1917–1922 (IEEE, 2017).
39. Yang, L., Ciptadi, A., Laziuk, I., Ahmadzadeh, A. & Wang, G. Bodmas: An open dataset for learning based temporal analysis of pe malware. In *Proceedings of the 4th Deep Learning and Security Workshop* (2021).
40. Bensaoud, A., Abudawaood, N. & Kalita, J. Classifying malware images with convolutional neural network models. *Int. J. Netw. Secur.* **22**, 1022–1031 (2020).
41. Karanja, E. M., Masupe, S. & Jeffrey, M. G. Analysis of internet of things malware using image texture features and machine learning techniques. *Internet Things* **9**, 100153 (2020).